

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Інститут дистанційної і заочної освіти
Кафедра інженерії програмного забезпечення

Аліна СКРИПНІКОВА

(група ЗАС-211)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

Вебзастосунок для побудови моделі розробника програмного забезпечення

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо - професійна програма:

Інженерія програмного забезпечення

Керівник:

Олексій КУНГУРЦЕВ, к.т.н., професор

Одеса – 2026

АНОТАЦІЯ

Скрипнікова А. А. Вебзастосунок для побудови моделі розробника програмного забезпечення : кваліфікаційна робота бакалавра за спеціальністю «121 Інженерія програмного забезпечення» / Аліна Андріївна Скрипнікова ; керівник Олексій Борисович Кунгурцев – Одеса : Нац. ун-т «Одес. політехніка», 2026. – 90 с.

Кваліфікаційна робота містить основну текстову частину на 74 сторінках, список використаних джерел з 12 найменувань на 2 сторінках, додатки на 9 сторінках.

Метою кваліфікаційної роботи є скорочення часу і підвищення ефективності підбору розробників для виконання програмних завдань шляхом автоматизації процесу оцінювання відповідності кандидатів вимогам задачі.

У роботі розроблено вебзастосунок, який дозволяє зберігати інформацію про розробників (навички, досвід, спеціалізації, освіту) та формувати задачі з відповідними вимогами. Реалізовано алгоритм підбору кандидатів, який виконує оцінку ступеня відповідності розробників вимогам задачі та формує ранжований список найбільш підходящих кандидатів.

Система реалізована із використанням технологій .NET, Entity Framework Core, SQLite та сучасних вебтехнологій. У процесі розробки було проведено аналіз предметної області та аналогів, сформовано функціональні та нефункціональні вимоги, спроектовано архітектуру системи, базу даних та користувацький інтерфейс.

Результатом кваліфікаційної роботи є вебзастосунок для підбору розробників, який може бути використаний для автоматизації процесів рекрутингу та управління ресурсами в ІТ-проектах. Проведене тестування підтвердило коректність роботи системи та відповідність поставленим вимогам.

Ключові слова: вебзастосунок, підбір розробників, алгоритм підбору, оцінка кандидатів, база даних, .NET.

ABSTRACT

Skripnikova A. A. Web application for a custom software developer model : Bachelor's qualification work in the specialty «121 Software Engineering» / Alina Andreyevna Skripnikova ; supervisor Oleskii Borysovykh Kungurtsev – Odesa: Odesa Polytech. Nat. Univ., 2026. – 90 pages.

The qualification work contains the main text part on 74 pages, a list of used sources with 12 titles on 2 pages, and appendices on 9 pages.

The aim of qualification work is to reduce time and increase the efficiency of selecting developers to perform software tasks by automating the process of assessing candidates' compliance with the task requirements.

The work presents a web application that allows storing information about developers (skills, experience, specializations, education) and creating tasks with defined requirements. A candidate matching algorithm is implemented, which evaluates the degree of compliance of developers with task requirements and generates a ranked list of the most suitable candidates.

The system is implemented using .NET, Entity Framework Core, SQLite and modern web technologies. During the development process, the subject area and existing solutions were analyzed, functional and non-functional requirements were defined, and the system architecture, database, and user interface were designed.

The result of the qualification work is a web application for developer selection, which can be used to automate recruitment processes and resource management in IT projects. Testing confirmed the correctness of the system and its compliance with the specified requirements.

Keywords: web application, developer selection, matching algorithm, candidate evaluation, database, .NET.

ЗМІСТ

Вступ.....	6
1 Специфікація вимог до вебзастосунку.....	8
1.1 Опис предметної області	8
1.2 Аналіз аналогів	9
1.3 Функціональні вимоги	13
1.4 Нефункціональні вимоги	14
1.5 Вимоги до інтерфейсу користувача	15
1.6 Варіанти використання системи	16
Висновки до розділу 1	20
2 Планування проекту з розробки вебзастосунку.....	22
2.1 Оцінка масштабів проекту.....	22
2.2 Оцінка ризиків	26
Висновки до розділу 2	29
3 Алгоритм підбору розробників.....	31
3.1 Основні положення	31
3.2 Формування вимог до задачі.....	31
3.3 Алгоритм попередньої фільтрації кандидатів.....	32
3.4 Алгоритм оцінювання відповідності розробників.....	34
3.5 Алгоритм обчислення рейтингу кандидатів.....	37
Висновки до розділу 3	41
4 Проєктування вебзастосунку	43
4.1 Проєктування моделі розробника та задачі.....	43
4.2 Архітектура вебзастосунку	46

4.3 Проєктування структури бази даних.....	48
Висновки до розділу 4	50
5 Програмна реалізація вебзастосунку	52
5.1 Вибір інструментів розробки	52
5.2 Реалізація сервісу підбору кандидатів	53
Висновки до розділу 5	56
6 Тестування вебзастосунку	57
6.1 Тестування функціональних вимог	57
6.2 Тестування нефункціональних вимог	59
Висновки до розділу 6	61
7 Експлуатація застосунку	62
7.1 Робота користувача з системою.....	62
7.2 Приклад підбору кандидатів для програмної задачі	72
Висновки до розділу 7	77
Загальні висновки.....	78
Список використаних джерел	80
Додаток А Програмна реалізація моделі Developer.....	82
Додаток Б програмна реалізація моделі taskentity	83
Додаток В Конфігурація контексту бази даних AppDbContext	84
Додаток Г Реалізація сервісу підбору кандидатів.....	85

ВСТУП

Актуальність. Традиційні методи підбору розробників, що базуються на ручному аналізі резюме та суб'єктивній оцінці, часто є трудомісткими, затратними за часом і не завжди дозволяють об'єктивно визначити найбільш підходящого кандидата. У зв'язку з цим виникає необхідність створення програмних систем, здатних автоматизувати процес зіставлення вимог задачі з характеристиками розробників, що обумовлює потребу у спеціалізованих програмних рішеннях.

Актуальність роботи зумовлена такими факторами:

- зростанням кількості ІТ-проектів і потребою в ефективному підборі спеціалістів;
- необхідністю скорочення часу пошуку відповідних кандидатів;
- потребою в об'єктивному оцінюванні розробників на основі множини параметрів;
- розвитком вебтехнологій і можливостей автоматизації процесів прийняття рішень.

Темою даної кваліфікаційної роботи є розробка вебзастосунку для підбору розробників під конкретні задачі на основі аналізу їх професійних характеристик і вимог проекту.

У межах роботи передбачається розробка системи, що дозволяє зберігати інформацію про розробників, їх навички, досвід, спеціалізації та освіту, а також формувати задачі з набором вимог і автоматично визначати ступінь відповідності кандидатів.

Метою роботи є підвищення ефективності процесу підбору розробників за рахунок скорочення часу підбору кандидатів.

Для досягнення поставленої мети необхідно вирішити такі задачі:

- дослідити предметну область підбору розробників;
- провести аналіз існуючих рішень і підходів;
- сформулювати функціональні та нефункціональні вимоги до системи;
- спроектувати архітектуру застосунку, структуру бази даних і основні

компоненти системи;

- розробити вебзастосунок із використанням сучасних технологій;
- розробити алгоритм підбору та оцінювання кандидатів;
- провести тестування функціональних і нефункціональних вимог системи.

Об’єктом дослідження є процес створення вебзастосунку для підбору розробників з метою виконання програмних задач.

Предметом дослідження є розробка та реалізація вебзастосунку, що забезпечує автоматизований підбір і оцінювання розробників на основі заданих критеріїв.

1 СПЕЦИФІКАЦІЯ ВИМОГ ДО ВЕБЗАСТОСУНКУ

1.1 Опис предметної області

У сучасних умовах розробки програмного забезпечення процес підбору розробників є складною та багатокритеріальною задачею. Для ефективного виконання програмних задач необхідно враховувати різні характеристики кандидатів, зокрема професійні навички, рівень їх володіння, досвід роботи, спеціалізацію, а також знання певної предметної області [1, 2].

У наукових дослідженнях підкреслюється важливість як технічних, так і комунікаційних компетенцій розробників для успішної роботи в проєктах [1]. При цьому вибір кандидата часто залежить не лише від наявності окремих навичок, а й від їх відповідності конкретним вимогам задачі [2].

Процес підбору розробників у більшості випадків здійснюється на основі аналізу резюме, портфоліо, результатів співбесід та попереднього досвіду роботи. Однак такий підхід значною мірою залежить від суб'єктивної оцінки, що може призводити до неточностей при прийнятті рішень.

Крім того, сучасні програмні проєкти характеризуються високою складністю та різноманітністю вимог. Це потребує врахування великої кількості параметрів одночасно, що ускладнює процес аналізу кандидатів та підвищує ризик помилок при їх відборі [2].

Існуючі підходи до підбору виконавців включають використання різних моделей оцінювання та автоматизованих методів аналізу характеристик розробників. Зокрема, пропонуються підходи, що базуються на формалізації вимог задачі та порівнянні їх із характеристиками кандидатів [3, 4].

У наукових дослідженнях також розглядаються підходи до підбору розробників.

В роботі [1] наведено деталізацію навичок та компетенцій розробників програмного забезпечення (ПЗ), зазначаючи важливість як технічних компетенцій так і комунікаційних навичок для успішної роботи в проєктах. Проте, автори не

розглядають відповідність компетенцій спеціаліста конкретному завданню.

В дослідженні [2] пропонується модель завдання на розробку ПЗ, але відсутня розгорнута модель виконавця завдання.

В роботі [3] розроблено метод побудови моделі спеціаліста на основі автоматизованого аналізу документів, в створенні яких він приймав участь. Такий підхід може бути ефективним за умови накопичення достатньої кількості документів.

Таким чином, процес підбору розробників є складною задачею, що потребує застосування більш об'єктивних та автоматизованих підходів до оцінювання кандидатів.

1.2 Аналіз аналогів

У процесі дослідження предметної області підбору розробників доцільним етапом є аналіз існуючих аналогічних рішень, представлених на ринку. Такий аналіз дозволяє виявити ефективні підходи до реалізації систем підбору фахівців, а також визначити їхні недоліки з метою врахування у подальшій розробці системи. Особливу увагу під час аналізу було приділено таким аспектам:

- способу представлення інформації про кандидатів;
- методам фільтрації та пошуку розробників;
- наявності механізмів оцінювання відповідності кандидатів вимогам;
- зручності користувацького інтерфейсу;
- можливостям управління даними.

З урахуванням специфіки предметної області було обрано такі аналоги:

- платформа пошуку розробників Upwork [5];
- сервіс пошуку роботи LinkedIn [6];
- платформа для розробників GitHub [7].

Порівняння проводилося за критеріями наявності системи оцінювання кандидатів, гнучкості фільтрації, зручності інтерфейсу, а також можливості автоматизованого підбору фахівців.

Upwork – являє собою платформу для пошуку фрилансерів, зокрема розробників [5]. Користувач може публікувати задачі та обирати кандидатів на основі їхніх профілів, рейтингу та досвіду (рис 1.1).

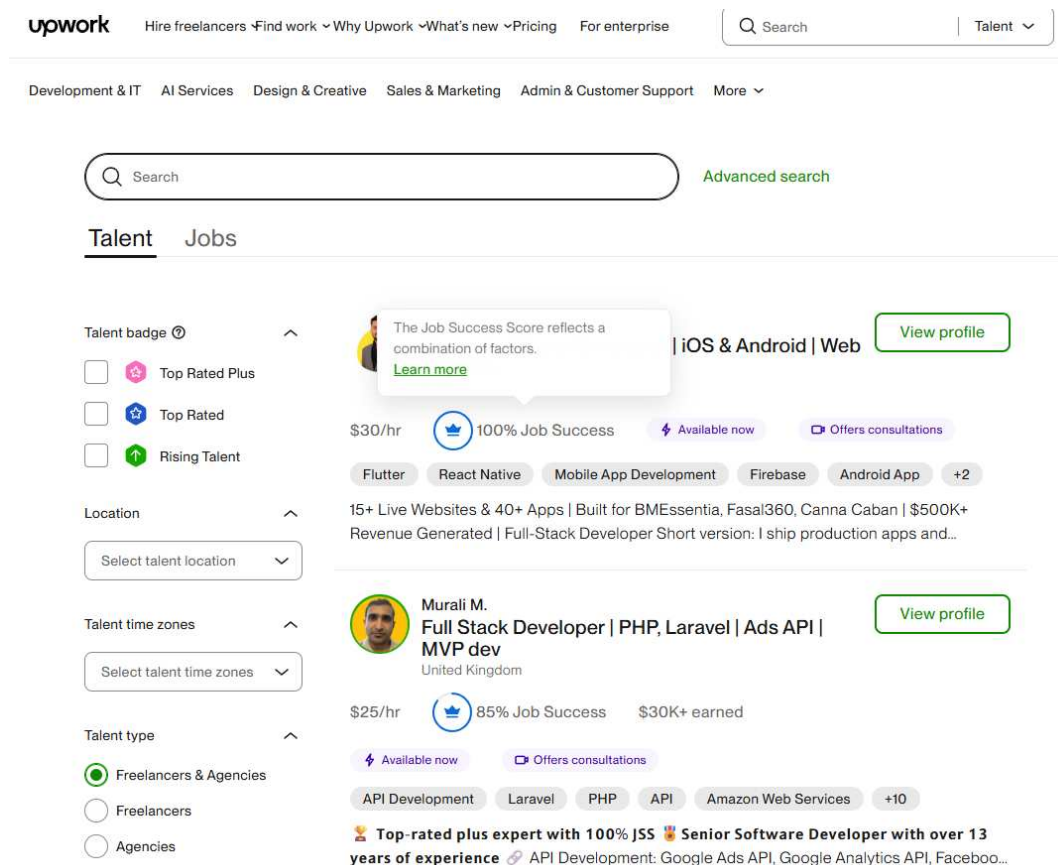


Рисунок 1.1 – Інтерфейс пошуку фахівців на платформі Upwork

Переваги:

- наявність рейтингів і відгуків;
- можливість фільтрації кандидатів;
- велика кількість фахівців.

Недоліки:

- відсутній формалізований алгоритм підбору;
- оцінювання кандидатів є суб'єктивним;
- відсутній облік ваг критеріїв.

LinkedIn – використовується для пошуку фахівців та аналізу їхнього професійного досвіду [6] (рис. 1.2).

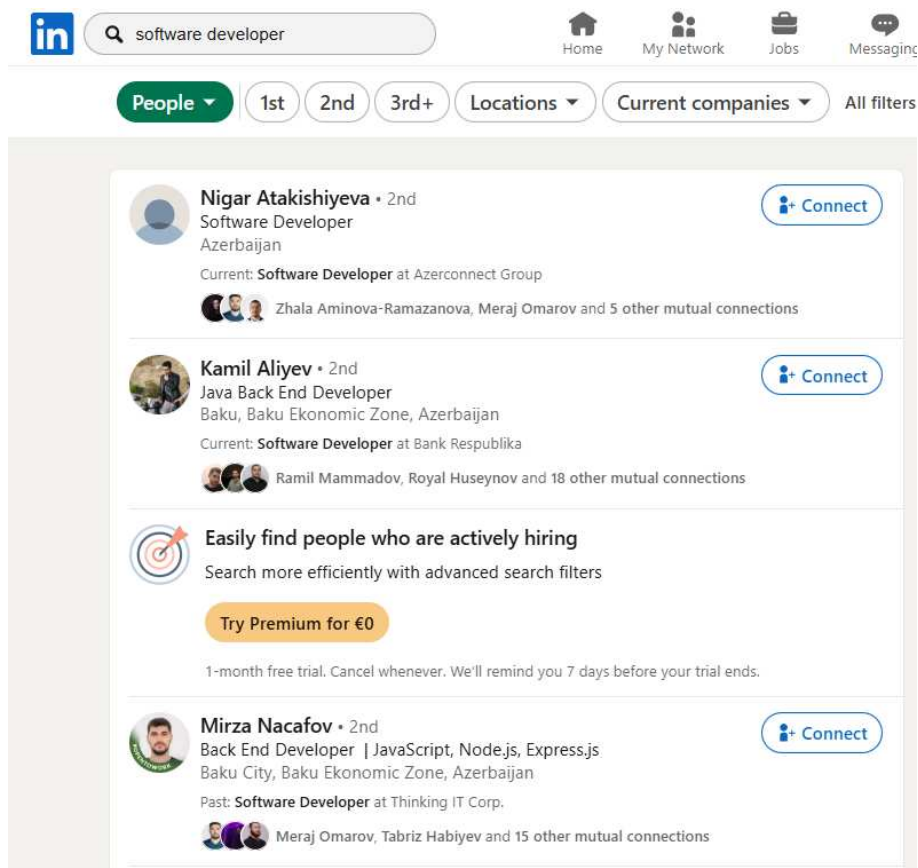


Рисунок 1.2 – Результати пошуку фахівців у LinkedIn

Переваги:

- детальні профілі кандидатів;
- зручний пошук і фільтрація;
- наявність рекомендацій.

Недоліки:

- відсутній автоматичний підбір;
- відсутня формальна модель оцінювання відповідності;
- підбір здійснюється вручну.

GitHub – дозволяє аналізувати активність розробників і їхні проекти [7] (рис. 1.3).

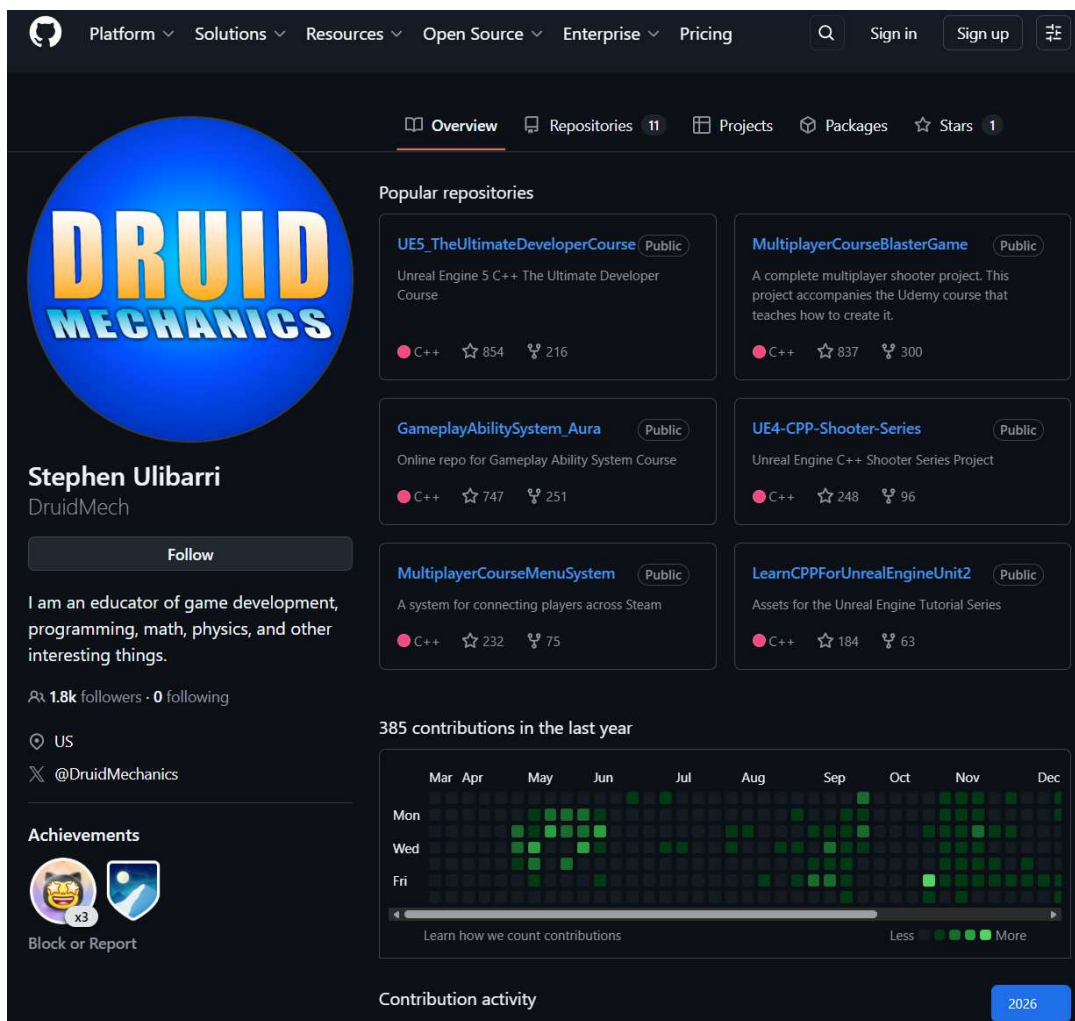


Рисунок 1.3 – Профіль розробника на GitHub

Переваги:

- доступ до реального коду;
- можливість оцінювання активності;
- відкриті проєкти.

Недоліки:

- відсутня система підбору кандидатів;
- не враховуються вимоги задачі;
- відсутній механізм ранжування.

Для більш наочного представлення результатів аналізу аналогів виконано їх порівняння за основними критеріями, такими як наявність автоматичного підбору, оцінювання кандидатів, гнучкість фільтрації та інші функціональні можливості. Результати порівняння наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння аналогів

Критерій	Upwork	LinkedIn	GitHub	Розроблений застосунок
Пошук розробників	+	+	+	+
Фільтрація	+	+	частково	+
Автоматичний підбір	—	—	—	+
Оцінювання кандидатів	частково	частково	частково	+
Облік ваг критеріїв	—	—	—	+
Ранжування кандидатів	—	—	—	+
Робота з задачами	—	—	—	+
Зручність інтерфейсу	+	+	+	+

1.3 Функціональні вимоги

Функціональні вимоги визначають основні можливості системи та дії, які може виконувати користувач.

У розроблюваному вебзастосунку передбачається реалізація таких функціональних можливостей:

- створення, редагування та видалення розробників;
- додавання та редагування професійних навичок розробників;

- управління ролями, спеціалізаціями та предметними областями;
- створення, редагування та видалення програмних задач;
- задання вимог до кандидатів для кожної задачі;
- автоматичний підбір кандидатів для обраної задачі;
- перегляд списку кандидатів із зазначенням рейтингу;
- перегляд детальної інформації про результати оцінювання кандидатів.

1.4 Нефункціональні вимоги

Нефункціональні вимоги визначають характеристики якості розроблюваної системи та умови її функціонування.

До основних нефункціональних вимог вебзастосунку належать:

- продуктивність: система повинна забезпечувати час відгуку не більше 2 секунд при виконанні основних операцій, зокрема при розрахунку рейтингу кандидатів і формуванні списку результатів;
- зручність використання: інтерфейс застосунку має бути інтуїтивно зрозумілим, забезпечувати просту навігацію між розділами та мінімізувати кількість дій, необхідних для виконання основних операцій; у системі передбачається використання підказок до полів введення, пояснень до параметрів інтерфейсу (tooltips), а також інструкцій для користувача, що пояснюють призначення основних функцій системи;
- надійність: система повинна забезпечувати стабільну роботу з рівнем доступності не менше 99% часу, коректно обробляти користувацькі дані та запобігати виникненню критичних помилок при роботі з базою даних;
- масштабованість: архітектура застосунку повинна забезпечувати можливість розширення функціональності, зокрема додавання нових критеріїв оцінювання кандидатів і додаткових сутностей без суттєвих змін існуючої системи;
- підтримуваність: програмний код має бути структурованим і зрозумілим, що забезпечує можливість подальшої модифікації та супроводу системи;
- цілісність даних: система повинна забезпечувати коректність зберігання

даних і запобігати дублюванню записів.

1.5 Вимоги до інтерфейсу користувача

Інтерфейс вебзастосунку повинен бути інтуїтивно зрозумілим, логічно організованим і забезпечувати зручну взаємодію користувача із системою. Користувачем системи є тімлід – керівник команди розробників.

Особлива увага приділяється зручності навігації, наочності представлення даних і доступності основних дій користувача.

Для цих вимог у системі потрібно передбачити такі елементи інтерфейсу:

- сторінки управління сутностями системи (розробники, задачі, навички, ролі, спеціалізації та предметні області), які будуть організовані у вигляді окремих розділів для забезпечення зручної навігації та структурованого представлення даних;

- форми створення та редагування сутностей, що будуть містити динамічні елементи інтерфейсу, які дозволять додавати та видаляти поля введення (наприклад, навички розробника або вимоги задачі) без перезавантаження сторінки;

- механізми валідації даних, що запобігатимуть введенню некоректної інформації. У разі виникнення помилок користувачу будуть відображатися зрозумілі повідомлення (наприклад: «Поле є обов'язковим», «Таке значення вже існує»);

- механізм запобігання дублюванню даних, який забезпечить перевірку унікальності сутностей (навичок, ролей, спеціалізацій або предметних областей);

- підказки, що пояснюватимуть призначення параметрів інтерфейсу. Наприклад, під час створення задачі будуть відображатися пояснення для параметрів рівня і ваги, а під час перегляду результатів підбору пояснення показника перевищення рівня навичок;

- інтерфейс сторінки підбору кандидатів, який надаватиме користувачу наочну інформацію про вимоги задачі та результати оцінювання, а також міститиме

додаткові підказки для кращого розуміння структури розрахунку рейтингу кандидатів;

- механізми контролю цілісності даних при видаленні, які не дозволятимуть видаляти сутності, пов'язані з іншими об'єктами системи;

- єдиний стиль оформлення інтерфейсу з використанням сучасних вебтехнологій (HTML, CSS, Bootstrap), що забезпечить візуальну узгодженість і зручність сприйняття інформації;

- адаптивний інтерфейс, що забезпечить коректну роботу системи на пристроях із різними розмірами екрана.

1.6 Варіанти використання системи

Варіанти використання системи дозволяють описати основні сценарії взаємодії користувача із вебзастосунком та відобразити функціональні можливості системи з точки зору користувача.

Діаграма варіантів використання відображає основні операції, які повинні бути доступні користувачу, та дозволяє визначити межі системи, а також ключові сценарії її роботи.

У розроблюваному вебзастосунку підбору розробників основним актором є користувач (тімлід, керівник команди розробників), який здійснює взаємодію із системою, управляє даними та ініціює процес підбору кандидатів.

Робота тімліда із системою має послідовний характер: спочатку здійснюватимуться введення даних про розробників, далі формується задача із відповідними вимогами, після чого виконується підбір кандидатів і відображаються результати. Зазначена послідовність дій детально представлена у сценаріях 1.1–1.3.

На діаграмі варіантів використання представлені основні дії користувача (тімліда) та їх взаємозв'язок у межах роботи системи (рис. 1.4).



Рисунок 1.4 – Діаграма варіантів використання системи

Сценарій 1.1 – Додавання нового розробника

Мета: додати нового розробника до системи з урахуванням його професійних характеристик.

Основна діюча особа: тімлід (Team Lead).

Область дії: вебзастосунок підбору розробників.

Рівень: користувацький.

Передумови: тімлід має доступ до системи.

Тригер: тімлід обирає функцію «Створити розробника» («Create Developer»).

Основний сценарій:

1. Тімлід обирає функцію «Створити розробника» («Create Developer»).
2. Система відображає форму для введення даних про розробника.
3. Тімлід вводить основну інформацію про розробника (ПІБ (FullName), email, поточну роль (Current Role), кількість років досвіду (Experience Years), рівень освіти (Education level)).
4. Тімлід обирає спеціалізації розробника (Specialties).
5. Тімлід обирає предметні області (Domain).
6. Тімлід обирає попередні ролі розробника (Previous Roles).
7. Тімлід додає професійні навички (Skills) із зазначенням рівня володіння.

8. Тімлід натискає кнопку «Зберегти» («Save»).
9. Система перевіряє коректність введених даних.
10. Система перевіряє унікальність email у базі даних.
11. Система зберігає дані нового розробника у базі даних.

Розширення:

- 9а. Якщо введені дані некоректні або не заповнені обов'язкові поля, система виводить повідомлення про помилку та не виконує збереження.
- 10а. Якщо у системі вже існує розробник із таким email, система виводить повідомлення про помилку «Розробник з таким email вже існує» («Developer with this email already exists») та не зберігає дані.

Сценарій 1.2 – Створення задачі

Мета: створити нову задачу із заданими вимогами до кандидата для подальшого підбору розробників.

Основна діюча особа: тімлід (Team Lead).

Область дії: вебзастосунок підбору розробників.

Рівень: користувацький.

Передумови: тімлід має доступ до системи.

Тригер: тімлід обирає функцію «Створити задачу» («Create Task»).

Основний сценарій:

1. Тімлід обирає функцію «Створити задачу» («Create Task»).
2. Система відображає форму створення задачі.
3. Тімлід вводить основну інформацію про задачу (назву (Title), опис (Description), тип задачі (Task Type), пріоритет (Priority), тривалість виконання (DurationDays)).
4. Тімлід вказує необхідний досвід (Required Experience (years)) та задає вагу цього параметра (ExperienceWeight).
5. Тімлід обирає предметну область (Domain) та задає її вагу (DomainWeight).
6. Тімлід обирає необхідний рівень освіти (Required Education Level) та задає вагу (EducationWeight).

7. Тімлід додає ролі (Roles) та визначає їх важливість для задачі.
8. Тімлід додає спеціалізації (Specialities) та визначає їх важливість.
9. Тімлід додає вимоги до навичок (Skill Requirements), вказуючи рівень володіння та вагу кожної навички.
10. Тімлід натискає кнопку «Зберегти» («Save»).
11. Система перевіряє коректність введених даних.
12. Система зберігає задачу та її вимоги у базі даних.

Розширення:

- 9а. Якщо тімлід не додав жодної обов'язкової навички (Skill Requirements), система виводить повідомлення про необхідність додати хоча б одну навичку.
- 11а. Якщо не заповнені обов'язкові поля (наприклад, назва задачі або вимоги до навичок), система виводить повідомлення про помилку та не зберігає задачу.
- 11б. Якщо введені значення параметрів (ваги або рівні) виходять за допустимі межі, система виводить повідомлення про помилку та пропонує виправити дані.

Сценарій 1.3 – Підбір розробників для задачі

Мета: визначити найбільш відповідних розробників для виконання задачі на основі заданих вимог та критеріїв.

Основна діюча особа: тімлід (Team Lead).

Область дії: вебзастосунок підбору розробників.

Рівень: користувацький.

Передумови: у системі наявні дані про розробників; створено задачу з вимогами.

Тригер: тімлід натискає кнопку «Знайти кандидатів» («Find Candidates») для обраної задачі.

Основний сценарій:

1. Тімлід переглядає список задач.
2. Тімлід обирає потрібну задачу.
3. Тімлід натискає кнопку «Знайти кандидатів» («Find Candidates»).
4. Система відкриває сторінку з результатами підбору кандидатів.
5. Система відображає інформацію про задачу та її вимоги (тип, пріоритет, досвід,

освіта, домен, ролі, спеціалізації, навички).

6. Система відображає пояснення параметрів оцінювання (level – рівень володіння навичкою, weight – важливість параметра).
7. Система застосовує правило відбору: кандидати без жодної відповідної навички виключаються з процесу підбору.
8. Система отримує список розробників із бази даних.
9. Система виконує попередню фільтрацію розробників відповідно до обов'язкових навичок.
10. Система обчислює ступінь відповідності кожного розробника вимогам задачі з урахуванням навичок, досвіду, освіти, домену, ролей та ваг параметрів.
11. Система розраховує підсумковий рейтинг (Score) для кожного кандидата.
12. Система формує список кандидатів, відсортований за значенням рейтингу.
13. Система відображає результати у вигляді таблиці (позиція, ім'я розробника, рейтинг, досвід, освіта, поточна роль, показник skill surplus).
14. Тімлід переглядає список кандидатів.
15. Тімлід натискає кнопку «Деталі» («Details») для обраного кандидата.
16. Система відображає детальну інформацію про відповідність кандидата вимогам задачі.
17. Тімлід аналізує результати та приймає рішення щодо вибору кандидата.

Розширення:

- 7а. Якщо жоден розробник не має жодної відповідної навички, система виводить повідомлення про відсутність кандидатів.
- 9а. Якщо у системі відсутні розробники, система виводить повідомлення про відсутність кандидатів для підбору.

Висновки до розділу 1

У першому розділі було виконано аналіз вимог підбору розробників для виконання програмних задач, який показав, що даний процес є складним і багатокритеріальним та потребує врахування значної кількості характеристик

кандидатів, зокрема їх професійних навичок, досвіду, спеціалізації та відповідності вимогам конкретної задачі.

Проведений аналіз наукових джерел дозволив встановити, що існуючі підходи до оцінювання розробників або зосереджуються на загальних компетенціях спеціалістів, або не враховують повною мірою відповідність кандидата конкретному завданню. Це підтверджує необхідність розробки більш формалізованих і об'єктивних методів підбору виконавців.

У результаті аналізу аналогів було визначено, що існуючі системи не забезпечують повноцінного автоматизованого підбору кандидатів, не враховують вагу критеріїв та не реалізують механізм ранжування розробників на основі комплексної оцінки. Виявлені недоліки стали підґрунтям для формування вимог до розроблюваного вебзастосунку.

На основі проведеного аналізу було сформовано функціональні вимоги до системи, які визначають основні можливості вебзастосунку, зокрема управління даними про розробників, створення задач, задання вимог до кандидатів та автоматизований підбір і ранжування виконавців.

Також були визначені нефункціональні вимоги, що характеризують якість системи, включаючи вимоги до продуктивності, надійності, зручності використання, масштабованості та цілісності даних.

Окрему увагу приділено вимогам до інтерфейсу користувача, які передбачають створення інтуїтивно зрозумілого, зручного та адаптивного інтерфейсу з використанням сучасних вебтехнологій.

Крім того, у розділі було розглянуто варіанти використання системи та сформовано основні сценарії взаємодії користувача (тімліда) із вебзастосунком, що дозволило деталізувати функціональні можливості системи та визначити логіку її роботи.

Таким чином, у першому розділі було сформовано повну специфікацію вимог до вебзастосунку підбору розробників, яка є основою для подальшого проектування та реалізації системи.

2 ПЛАНУВАННЯ ПРОЕКТУ З РОЗРОБКИ ВЕБЗАСТОСУНКУ

2.1 Оцінка масштабів проекту

Під час розробки вебзастосунку для підбору розробників важливим етапом є попередня оцінка обсягу робіт і планування проекту. Це дозволяє визначити послідовність виконання задач, розподілити навантаження за етапами та забезпечити своєчасне завершення проекту.

Для вирішення цієї задачі розробку системи було поділено на окремі функціональні етапи (табл. 2.1). Такий підхід дозволяє більш точно оцінити трудомісткість проекту, а також враховувати особливості реалізації різних компонентів системи, включаючи користувацький інтерфейс, бізнес-логіку та алгоритми підбору кандидатів.

На основі аналізу структури системи було сформовано ієрархічну структуру робіт, що охоплює всі ключові етапи розробки (таб. 2.1).

Таблиця 2.1 – Ієрархічна структура робіт

№	Етап	Опис
1	2	3
1	Дослідження предметної області та постановка задачі	Аналіз існуючих рішень, визначення мети системи та формування вимог
2	Проектування структури даних	Визначення сутностей (Developer, Task, Skills тощо) та зв'язків між ними
3	Розробка архітектури застосунку	Визначення структури системи та поділ на рівні (інтерфейс, логіка, дані)
4	Розробка користувацького інтерфейсу	Створення сторінок, форм введення даних, реалізація динамічних елементів

Кінець таблиці 2.1

1	2	3
5	Реалізація функціональності розробників	CRUD-операції, управління навичками, ролями, спеціалізаціями
6	Реалізація функціональності задач	Створення задач і налаштування вимог до кандидатів
7	Розробка алгоритму підбору	Реалізація MatchingService і методів оцінювання кандидатів
8	Інтеграція компонентів системи	Об'єднання всіх модулів і налаштування взаємодії
9	Тестування системи	Перевірка коректності роботи алгоритмів та інтерфейсу
10	Підготовка документації	Оформлення пояснювальної записки та підготовка до захисту

Розглянемо основні етапи розробки більш детально.

Дослідження предметної області (03.02 – 07.02):

- аналіз існуючих рішень підбору кандидатів;
- визначення основні критерії оцінювання розробників;
- формування вимоги до системи.

Проектування структури даних (08.02 – 12.02):

- проектування моделі бази даних;
- визначення основних сутностей та зв'язки;
- реалізація структури за допомогою Entity Framework Core.

Розробка архітектури застосунку (13.02 – 18.02):

- визначення багаторівневу архітектури;
- виділення рівнів системи;
- проектування сервісу MatchingService.

Розробка користувацького інтерфейсу (19.02 – 26.02):

- реалізація форми та сторінки застосунку;
- додавання інтерактивних елементів та валідації;
- реалізація підказок та динамічну поведінку інтерфейсу.

Реалізація функціональності розробників (27.02 – 05.03):

- реалізація операції створення, редагування та видалення;
- додавання управління пов'язаними сутностями.

Реалізація функціональності задач (06.03 – 12.03):

- реалізація створення задач;
- додавання налаштування вимог до кандидатів.

Розробка алгоритму підбору (13.03 – 17.03):

- реалізація методів CalculateScore та FindBestCandidatesAsync;
- додавання розрахунків додаткових показників.

Інтеграція системи (18.03 – 20.03):

- об'єднання всіх компонентів;
- забезпечення коректну взаємодію модулів.

Тестування та налагодження (21.03 – 22.03):

- перевірка алгоритма підбору;
- усунення виявлених помилок.

Оформлення пояснювальної записки (22.03 – 27.03):

- підготовка тексту дипломної роботи;
- оформлення таблиці, малюнків та схем.

Таблиця 2.2 – Календарний план виконання проєкту

№	Етап	Дати	Тривалість
1	2	3	4
1	Дослідження предметної області	03.02 – 07.02	5 днів
2	Проектування структури даних	08.02 – 12.02	5 днів

Кінець таблиці 2.2

1	2	3	4
3	Розробка архітектури застосунку	13.02 – 18.02	6 днів
4	Розробка користувацького інтерфейсу	19.02 – 26.02	8 днів
5	Реалізація функціональності розробників	27.02 – 05.03	7 днів
6	Реалізація функціональності задач	06.03 – 12.03	7 днів
7	Розробка алгоритму підбору	13.03 – 17.03	5 днів
8	Інтеграція системи	18.03 – 20.03	3 дні
9	Тестування та налагодження	21.03 – 22.03	2 дні
10	Оформлення документації	23.03 – 27.03	5 днів

Для визначення загального обсягу робіт застосовано метод функціональних точок (Function Point Analysis), що дозволяє оцінити складність системи на основі реалізованих функцій.

Кожному етапу розробки було присвоєно умовну оцінку складності (FP), що відображає його внесок у загальний обсяг робіт.

$$\Sigma FP = 5 + 5 + 6 + 8 + 7 + 7 + 5 + 3 + 2 + 5 = 53$$

Загальна трудомісткість проєкту визначається за формулою 2.1:

$$T = \Sigma FP \times H \quad (2.1)$$

де FP – кількість функціональних точок;
 H – середній час на одну точку (8 годин).

$$T = 53 \times 8 = 424 \text{ людино-годин}$$

Таким чином, сумарна трудомісткість розробки вебзастосунку становить близько 424 людино-годин, що відповідає проєкту середньої складності та включає всі етапи – від аналізу вимог до підготовки документації.

Для візуалізації календарного плану розробки було побудовано діаграму Ганта, що відображає послідовність і тривалість виконання етапів проєкту (рис. 2.1).

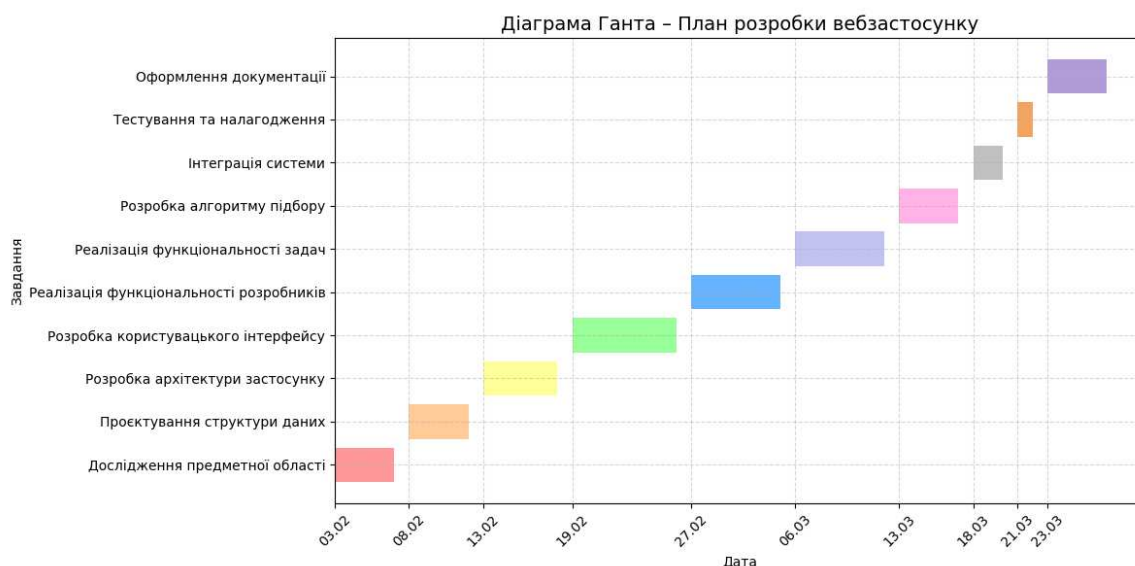


Рисунок 2.1 – Діаграма Ганта розробки вебзастосунку

2.2 Оцінка ризиків

У процесі розробки вебзастосунку для підбору розробників було проведено аналіз потенційних ризиків, які можуть вплинути на коректність роботи системи, якість користувацького інтерфейсу, а також на строки виконання проєкту.

Для оцінювання ризиків було використано комплексний підхід, що включає класифікацію ризиків за їх характером та аналіз можливих наслідків їх виникнення. Усі ризики було умовно поділено на кілька категорій: технічні, функціональні,

організаційні та ризики, пов'язані з людським фактором (таб. 2.3).

Таблиця 2.3 – Класифікація ризиків проєкту

Тип ризику	Приклади
Технічні	помилки в алгоритмі підбору, некоректні обчислення рейтингу, збої при роботі з базою даних
Функціональні	некоректна обробка вимог задачі, помилки у валідації даних
Організаційні	нестача часу на реалізацію або оформлення роботи, зміна вимог
Людський фактор	втома, неуважність, помилки під час тестування

Найбільш критичними є організаційні ризики, оскільки вони можуть опосередковано впливати на всі інші аспекти розробки. Недостатнє планування часу може призвести до скорочення етапу тестування, що підвищує ймовірність появи помилок у системі.

Обмежені строки виконання проєкту можуть зумовити необхідність прискореної розробки, що, у свою чергу, призводить до зниження якості коду та збільшення ймовірності логічних помилок. Крім того, зміна вимог у процесі розробки може спричинити необхідність переробки вже реалізованих компонентів системи.

Також важливим фактором є поєднання розробки проєкту з навчальною діяльністю, що зменшує доступний час і збільшує навантаження. Це може призвести до зниження концентрації та появи помилок на фінальних етапах розробки (таб 2.4).

Для мінімізації зазначених ризиків необхідно вживати наступних заходів:

- планування етапів розробки з використанням діаграми Ганта;
- пріоритизація задач і поетапна реалізація функціональності;
- рання реалізація ключових компонентів (алгоритму підбору);

- регулярна перевірка проміжних результатів;
- поступове оформлення документації в процесі розробки.

Таблиця 2.4 – Основні ризики реалізації системи підбору розробників

Назва ризику	Причина	Наслідки	Способи мінімізації
Помилка в алгоритмі підбору	некоректна реалізація формул	неправильний рейтинг кандидатів	тестування алгоритму на прикладах
Некоректна валідація даних	відсутність або неповна перевірка	помилки в даних, некоректні розрахунки	подвійна валідація (frontend + backend)
Некоректна фільтрація кандидатів	помилка в логіці фільтрації	виключення відповідних кандидатів	перевірка умов фільтрації
Помилки при роботі з базою даних	неправильні зв'язки або запити	втрата або спотворення даних	використання Entity Framework Core
Проблеми користувацького інтерфейсу	відсутність підказок або незручний UI	користувач не розуміє систему	додавання tooltip і валідації
Зміна вимог	додавання нових умов	переробка системи	фіксація вимог на ранньому етапі
Нестача часу	обмежені строки	зниження якості реалізації	буфер часу та планування
Людський фактор	втома, неуважність	логічні та технічні помилки	повторна перевірка та тестування

Для додаткового оцінювання ризиків було використано метод SWOT-аналізу, що дозволяє виявити сильні та слабкі сторони проєкту, а також зовнішні можливості та загрози (таб. 2.5).

Таблиця 2.5 – SWOT-аналіз проєкту

Сильні сторони	Слабкі сторони
автономна система без зовнішніх API	реалізація виконується одним розробником
гнучка модель підбору кандидатів	висока складність алгоритму
можливість розширення системи	обмежений час на тестування
використання сучасних технологій (.NET, EF Core)	необхідність ручної перевірки результатів

Таким чином, проведений аналіз ризиків дозволив виявити потенційні проблеми на різних етапах розробки та визначити заходи щодо їх мінімізації. Це забезпечує більш стабільну реалізацію системи та підвищує надійність її роботи.

Висновки до розділу 2

У даному розділі роботи було виконано планування процесу розробки вебзастосунку для підбору розробників.

Проведено оцінювання масштабів проєкту, у межах якого визначено основні етапи розробки, виконано декомпозицію робіт і сформовано календарний план реалізації. Для візуалізації строків виконання задач було побудовано діаграму Ганта, що дозволяє наочно представити послідовність етапів розробки та розподіл часу.

Також виконано оцінювання трудомісткості проєкту на основі умовної кількості функціональних точок, що дозволило визначити загальний обсяг робіт і часові витрати на реалізацію системи.

У межах аналізу ризиків було виявлено потенційні проблеми, які можуть виникнути в процесі розробки та експлуатації системи, зокрема технічні,

функціональні, організаційні ризики та ризики, пов'язані з людським фактором. Для кожного з ризиків визначено можливі причини, наслідки та заходи щодо їх мінімізації.

Проведений SWOT-аналіз дозволив визначити сильні та слабкі сторони проєкту, а також виявити можливі загрози та переваги розроблюваної системи.

Таким чином, виконане планування та аналіз ризиків забезпечують більш ефективну організацію процесу розробки, дозволяють знизити ймовірність виникнення критичних помилок і сприяють успішній реалізації вебзастосунку.

3 АЛГОРИТМ ПІДБОРУ РОЗРОБНИКІВ

3.1 Основні положення

В умовах великої кількості розробників вибір відповідного кандидата для виконання конкретної задачі може бути досить складним. Тому виникає необхідність використання алгоритмів, які дозволяють автоматично аналізувати характеристики розробників і зіставляти їх із вимогами програмної задачі.

У розроблюваній інформаційній системі використовується алгоритм підбору розробників, який дозволяє визначити найбільш відповідних кандидатів для виконання конкретної задачі програмного проєкту. Алгоритм враховує характеристики задачі, професійні параметри розробників та виконує послідовний аналіз відповідності фахівців встановленим вимогам.

Процес підбору кандидатів у системі складатиметься з кількох послідовних етапів. Спочатку формуються вимоги задачі, потім виконується попередня фільтрація розробників, після чого проводиться оцінювання відповідності фахівців вимогам задачі. На основі отриманих результатів обчислюється рейтинг кандидатів та виконується їх ранжування.

Використання такого підходу дозволяє автоматизувати процес підбору фахівців і підвищити об'єктивність вибору розробників для виконання програмних задач.

3.2 Формування вимог до задачі

Першим етапом алгоритму підбору розробників є формування вимог до програмної задачі. На цьому етапі визначатиметься набір параметрів, які описують характеристики задачі та вимоги до розробника, необхідного для її виконання.

У розроблюваній системі вимоги до задачі формуватимуться на основі інформації, яку користувач вводить під час створення нової задачі. Користувачу потрібно вказати основні параметри задачі, що будуть дозволяти визначити вимоги до кандидата.

Окрім зазначення вимог до розробника, у системі також задаватиметься вага кожного параметра. Вага параметра показує ступінь важливості відповідного критерію під час підбору кандидатів. Використання ваг дозволяє враховувати, які вимоги є більш важливими для конкретної задачі.

У розроблюваному застосунку ваги можуть задаватися для таких параметрів:

- предметної області розробки;
- професійного досвіду розробника;
- рівня освіти;
- ролі розробника;
- спеціалізації;
- професійних навичок.

Використання вагових коефіцієнтів дозволяє більш гнучко оцінювати відповідність розробників вимогам задачі. На основі зазначених параметрів і їхніх ваг формується набір критеріїв, який використовується на наступних етапах алгоритму підбору кандидатів.

Таким чином, на етапі формування вимог створюється структурований опис задачі, який визначає набір критеріїв для подальшого аналізу кандидатів.

Кожен із заданих параметрів задачі відповідає певній характеристиці розробника та використовується на наступних етапах алгоритму для оцінювання ступеня відповідності кандидата вимогам задачі.

Задані значення параметрів, а також їхні вагові коефіцієнти будуть використовуватися під час обчислення показників відповідності та підсумкового рейтингу кандидатів. Це дозволить формалізувати процес підбору розробників і забезпечити можливість кількісної оцінки ступеня відповідності фахівців вимогам програмної задачі.

3.3 Алгоритм попередньої фільтрації кандидатів

Після формування вимог до задачі наступним етапом алгоритму підбору розробників є попередня фільтрація кандидатів. Основна мета цього етапу полягає

в тому, щоб виключити з подальшого аналізу тих розробників, які очевидно не підходять для виконання задачі.

У розроблюваної системі попередня фільтрація кандидатів виконується на основі вимог до професійних навичок розробника. На цьому етапі система аналізує навички, зазначені у вимогах задачі, та порівнює їх із навичками, якими володітимуть розробники, зареєстровані в системі.

Спочатку система отримуватиме список навичок, необхідних для виконання задачі. При цьому враховуватимуться лише ті навички, для яких задано додатну вагу. Навички з нульовою вагою розглядатимуться як додаткова інформація і не використовуються для фільтрації кандидатів.

Після цього система виконує перевірку розробників, що знаходяться в базі даних. Для кожного розробника аналізуватиметься набір його професійних навичок. Якщо розробник не володіє жодною з необхідних навичок задачі, він виключається з подальшого розгляду.

У випадку, якщо розробник володіє хоча б однією навичкою, зазначеною у вимогах задачі, він включається до списку кандидатів для подальшого аналізу. При цьому рівень володіння навичкою на етапі попередньої фільтрації не перевіряється. Розробники з нижчим рівнем володіння навичкою не виключаються, однак їх відповідність вимогам задачі враховується на наступному етапі алгоритму під час обчислення рейтингу.

Наприклад, якщо у вимогах задачі зазначено навички C# та SQLite, а розробник володіє лише навичкою C#, такий розробник включається до списку кандидатів для подальшого аналізу. Якщо ж розробник не володіє жодною з необхідних навичок, він виключається з розгляду.

Таким чином, на етапі попередньої фільтрації формується множина потенційних кандидатів, які відповідають базовим вимогам задачі. Це дозволяє значно скоротити обсяг подальших обчислень та підвищити ефективність роботи алгоритму підбору розробників.

Загальна послідовність виконання попередньої фільтрації кандидатів наведена на рисунку 3.1.

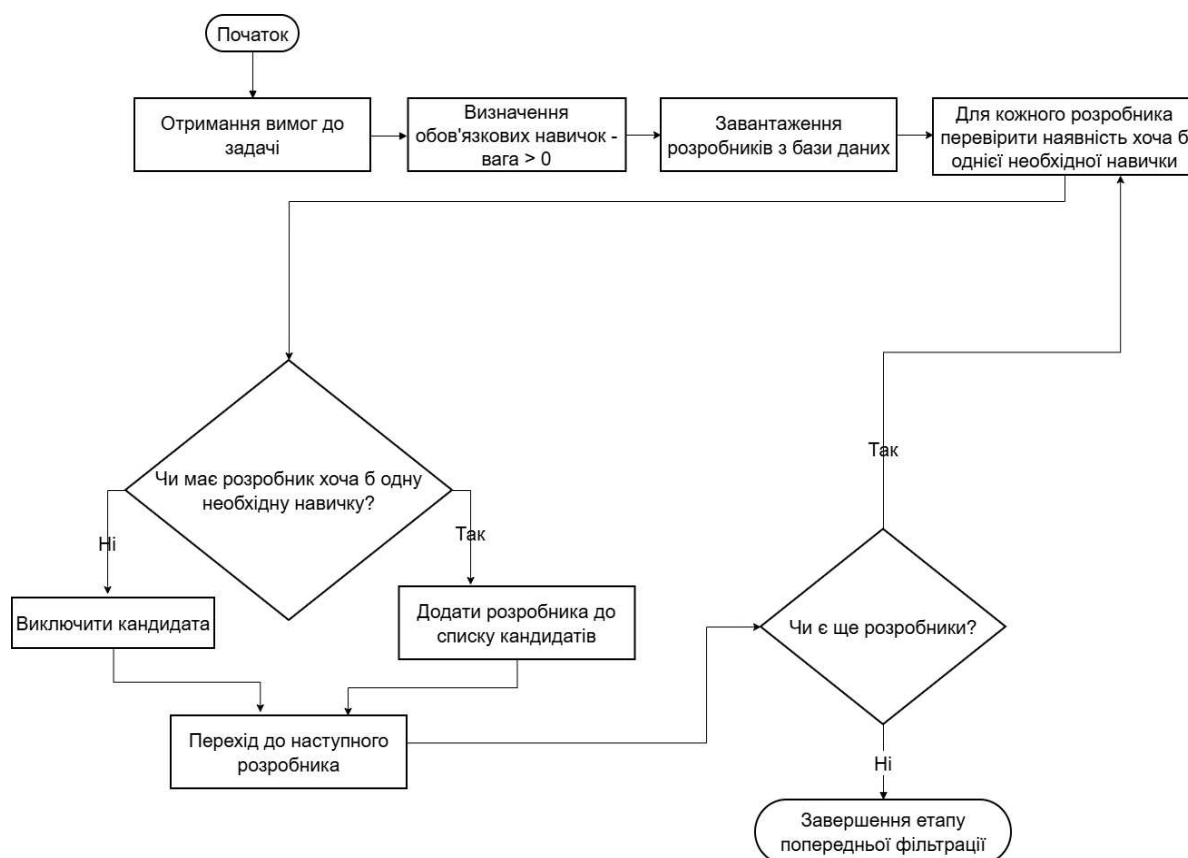


Рисунок 3.1 – Алгоритм попередньої фільтрації кандидатів

3.4 Алгоритм оцінювання відповідності розробників

Після виконання етапу попередньої фільтрації формується список розробників, які потенційно можуть бути обрані для виконання програмної задачі. На наступному етапі використовується більш детальний аналіз кожного кандидата та оцінюється ступінь його відповідності вимогам задачі.

Алгоритм оцінювання відповідності розробників призначений для визначення того, наскільки характеристики розробника збігаються з вимогами, зазначеними у задачі програмного проєкту.

Під час оцінювання враховуватимуться такі характеристики розробника:

- професійні навички;
- спеціалізація розробника;
- роль розробника у проєкті;
- предметна область розробки;
- рівень професійного досвіду;

– рівень освіти.

Для кожної характеристики виконується перевірка відповідності вимогам задачі. Залежно від ступеня збігу система надає певне значення показника відповідності.

Наприклад, під час оцінювання навичок розробника можуть використовуватися такі значення:

- 1 – повна відповідність необхідній навичці;
- значення у діапазоні від 0 до 1 – часткова відповідність (наприклад, наявність близької технології або схожого досвіду);
- 0 – відсутність необхідної навички.

У системі використовується аналогічний принцип і для інших характеристик розробника. Наприклад, під час оцінювання ролі розробника:

- 1 – роль повністю збігається з необхідною;
- 0 – роль не відповідає вимогам задачі.

Під час аналізу досвіду роботи значення показника може залежати від кількості років професійного досвіду. Наприклад:

- 1 – досвід повністю відповідає вимогам задачі;
- значення у межах від 0 до 1 – досвід менший за необхідний, але частково відповідає вимогам;
- 0 – досвід відсутній або значно нижчий за необхідний.

Для кожної необхідної навички обчислюється значення відповідності за формулою:

$$V_{skill} = \frac{X_{dev}}{X_{req}} \quad (3.1)$$

де V_{skill} – показник відповідності навички розробника вимогам задачі;

X_{dev} – рівень володіння відповідною навичкою у розробника;

X_{req} – необхідний рівень володіння навичкою, заданий у вимогах задачі.

Отримане значення обмежується діапазоном від 0 до 1. Якщо значення перевищує 1, воно обмежується до 1. У випадку, якщо розробник не володіє

необхідною навичкою, значення показника приймається рівним 0.

Наприклад, якщо для виконання задачі потрібен рівень володіння технологією, що дорівнює 5, а розробник має рівень 4, то:

$$V_{skill} = \frac{4}{5} = 0,8$$

Аналогічний підхід використовується для оцінювання досвіду роботи розробника. Значення показника визначається за формулою:

$$V_{exp} = \frac{Exp_{dev}}{Exp_{req}} \quad (3.2)$$

де V_{exp} – показник відповідності досвіду роботи розробника вимогам задачі;
 Exp_{dev} – фактичний досвід роботи розробника (у роках);
 Exp_{req} – необхідний досвід роботи, зазначений у вимогах задачі.

Наприклад, якщо для задачі необхідно 3 роки досвіду, а розробник має 2 роки, то:

$$V_{exp} = \frac{2}{3} \approx 0.67$$

Для інших характеристик, таких як роль розробника, спеціалізація та предметна область, використовуються дискретні значення відповідності. У цьому випадку:

- 1 – повна відповідність вимогам;
- 0 – відсутність відповідності.

Таким чином, для кожного розробника формується набір показників відповідності за окремими характеристиками. Ці показники будуть використовуватися для обчислення загальної оцінки відповідності кандидата

вимогам задачі.

Отримане значення будуть використовуватися на наступному етапі алгоритму для порівняння розробників між собою та формування списку кандидатів, відсортованого за ступенем відповідності вимогам задачі. Загальна послідовність виконання алгоритму оцінювання відповідності розробників наведена на рисунку 3.2.



Рисунок 3.2 – Алгоритм оцінювання відповідності розробників

3.5 Алгоритм обчислення рейтингу кандидатів

Після виконання етапу оцінювання відповідності розробників, описаного у підрозділі 3.3, для кожного кандидата формується набір показників відповідності. Ці показники відображають ступінь збігу характеристик розробника з вимогами задачі.

На попередньому етапі алгоритму для кожної характеристики розробника будуть обчислюватися значення відповідності. Значення відповідності показує, наскільки характеристики розробника задовольняють вимоги задачі. Воно визначається шляхом порівняння параметрів розробника з вимогами задачі та приймає значення у діапазоні від 0 до 1, де:

– 1 – повна відповідність вимогам задачі;

- $0 < \text{значення} < 1$ – часткова відповідність;
- 0 – відсутність відповідності.

Таким чином, значення відповідності характеризує результат порівняння характеристик розробника і вимог задачі.

Після визначення значень відповідності для всіх характеристик виконується обчислення підсумкового рейтингу кандидата.

Під час обчислення рейтингу враховуватимуться такі характеристики розробника:

- професійні навички;
- досвід розробки;
- предметна область розробки;
- рівень освіти;
- ролі розробника;
- спеціалізації розробника.

Для кожної характеристики задається вага, яка відображає її важливість під час підбору розробника для виконання задачі. Вага характеристики задається у вимогах задачі, оскільки різні задачі можуть вимагати різного ступеня важливості окремих характеристик.

Підсумковий рейтинг кандидата обчислюється з використанням зваженої суми показників відповідності за такою формулою:

$$Score = \frac{\sum_{i=1}^n (W_i \cdot V_i)}{\sum_{i=1}^n W_i} \quad (3.3)$$

де $Score$ – підсумковий рейтинг кандидата;

n – кількість характеристик, що беруть участь в оцінюванні;

W_i – вага i -тої характеристики, задана у вимогах задачі;

V_i – значення відповідності i -тої характеристики, отримане на етапі оцінювання відповідності розробника вимогам задачі.

Формула означає, що для кожної характеристики обчислюється добуток її

ваги на значення відповідності. Отримані значення підсумовуються, після чого результат ділиться на суму всіх ваг характеристик.

Такий підхід дозволяє враховувати різну важливість характеристик під час обчислення підсумкового рейтингу. Більш значущі характеристики мають більший вплив на підсумковий результат [4].

У результаті обчислення рейтинг приймає значення у діапазоні від 0 до 1, що дозволяє зручно порівнювати кандидатів між собою та виконувати їх ранжування.

Алгоритм обчислення рейтингу кандидата можна подати у вигляді такої послідовності дій:

- отримання значень відповідності характеристик розробника вимогам задачі;
- множення кожного значення відповідності на вагу відповідної характеристики;
- підсумовування отриманих значень;
- ділення отриманої суми на суму ваг характеристик;
- отримання підсумкового значення рейтингу кандидата;
- отримане значення рейтингу використовується на наступному етапі алгоритму для ранжування розробників і формування списку кандидатів, відсортованого за ступенем відповідності вимогам задачі.

Розглянемо приклад обчислення рейтингу розробника.

Припустимо, що для задачі програмного проєкту задано такі ваги характеристик: вага навичок дорівнює 5, вага досвіду роботи дорівнює 3, а вага відповідності предметної області дорівнює 2.

На попередньому етапі алгоритму було виконано перевірку відповідності характеристик розробника вимогам задачі. У результаті були отримані такі значення відповідності: відповідність навичок дорівнює 1, відповідність досвіду роботи дорівнює 0,7, а відповідність предметної області дорівнює 1.

Значення відповідності показує ступінь збігу характеристик розробника з вимогами задачі. Значення 1 означає повну відповідність, значення від 0 до 1 означає часткову відповідність, а значення 0 означає відсутність відповідності.

Після цього виконується обчислення внеску кожної характеристики у загальний рейтинг кандидата. Для цього значення ваги характеристики множиться на значення відповідності.

Таким чином отримуємо такі значення:

- для навичок: $5 \times 1 = 5$;
- для досвіду роботи: $3 \times 0,7 = 2,1$;
- для предметної області: $2 \times 1 = 2$.

Далі обчислюється загальний рейтинг кандидата. Для цього підсумовуються добутки ваги кожної характеристики на значення відповідності, після чого результат ділиться на суму всіх ваг характеристик.

У даному прикладі загальний рейтинг кандидата обчислюється таким чином:

$$\text{Score} = (5 \times 1 + 3 \times 0,7 + 2 \times 1) / (5 + 3 + 2).$$

Після виконання обчислень отримуємо:

$$\text{Score} = (5 + 2,1 + 2) / 10 = 0,91.$$

Таким чином, рейтинг даного кандидата становить 0,91. Отримане значення будуть використовуватися для порівняння кандидатів між собою. Чим вище значення рейтингу, тим краще розробник відповідає вимогам конкретної задачі програмного проєкту.

Під час ранжування кандидатів, окрім підсумкового рейтингу, у системі також використовується додатковий показник, що дозволяє більш точно враховувати рівень кваліфікації розробника.

Таким показником є значення перевищення рівня навичок розробника відносно вимог задачі (Skill Surplus).

Даний показник відображає, наскільки рівень володіння навичками у кандидата перевищує мінімально необхідний рівень, заданий у вимогах задачі.

Обчислення цього показника здійснюється за наступною формулою:

$$\text{Surplus} = \sum_{k=1}^m \max(0, \text{Level}_{dev,k} - \text{Level}_{req,k}) \quad (3.4)$$

де *Surplus* – сумарне перевищення рівня навичок розробника;

m – кількість навичок, що враховуються при обчисленні;

k – індекс навички (тобто порядковий номер навички у списку);

$Level_{dev,k}$ – рівень володіння k -ю навичкою у розробника;

$Level_{req,k}$ – необхідний рівень k -ї навички відповідно до вимог задачі.

Функція $\max(0, x)$ означає, що для кожної навички враховується лише додатна різниця між рівнем розробника та необхідним рівнем. У випадку, якщо різниця $Level_{dev,k} - Level_{req,k}$ є від'ємною або дорівнює нулю, її значення замінюється на 0 і не впливає на підсумковий результат.

Таким чином, для кожної навички виконується перевірка: якщо рівень розробника перевищує вимоги задачі, ця різниця додається до загального значення показника, якщо ж рівень є недостатнім, така навичка не робить внеску у показник Skill Surplus.

Нехай для певної навички необхідний рівень дорівнює 3, а рівень розробника становить 5. Тоді для однієї навички: $Surplus = 5 - 3 = 2$

Якщо ж рівень розробника дорівнює 2 при вимозі 3, то: $Surplus = 0$. Отримане значення показника використовується під час сортування кандидатів з однаковим або близьким значенням підсумкового рейтингу. Це дозволяє надавати перевагу тим розробникам, які не лише відповідають вимогам задачі, але й перевищують їх за рівнем кваліфікації.

Висновки до розділу 3

Розглянуто алгоритм підбору розробників для виконання програмних задач у розроблюваної інформаційній системі. Описано основні етапи роботи алгоритму та принципи аналізу відповідності характеристик розробників вимогам задачі.

У процесі дослідження було визначено підхід до формування вимог задачі, який включає такі характеристики, як професійні навички, спеціалізація, роль у проєкті, предметна область, рівень досвіду та освіти. Для кожної характеристики задається вага, що дозволяє враховувати її значущість при оцінюванні кандидатів.

Розглянуто етап попередньої фільтрації кандидатів, який дозволяє виключити

розробників, що не відповідають базовим вимогам, зокрема не мають необхідних навичок. Це дозволяє зменшити обсяг обчислень і підвищити ефективність роботи алгоритму.

Запропоновано метод оцінювання відповідності розробників вимогам задачі, який базується на порівнянні характеристик кандидата з параметрами задачі та визначенні ступеня їх відповідності у вигляді значень у діапазоні від 0 до 1.

На основі отриманих значень відповідності розглянуто обчислення підсумкового рейтингу кандидата з використанням зваженої формули. Це дозволяє об'єктивно оцінювати кандидатів з урахуванням важливості окремих характеристик. Додатково враховується показник перевищення рівня навичок (Skill Surplus), що дозволяє більш точно ранжувати кандидатів із близькими значеннями рейтингу.

У результаті створення алгоритму забезпечено можливість автоматизованого підбору та ранжування розробників відповідно до вимог програмної задачі. Запропонований підхід дозволяє формалізувати процес оцінювання кандидатів та підвищити об'єктивність прийняття рішень.

4 ПРОЄКТУВАННЯ ВЕБЗАСТОСУНКУ

4.1 Проєктування моделі розробника та задачі

У процесі проєктування вебзастосунок необхідно сформувати модель основних сутностей системи, зокрема модель розробника та модель програмної задачі. Дані моделі будуть використовуватися для формалізованого представлення інформації та подальшої реалізації алгоритму підбору кандидатів.

У розроблюваній системі розробник розглядається як основна сутність, що містить набір характеристик, необхідних для оцінювання його відповідності вимогам задачі.

Для формалізованого опису розробника визначатиметься такі основні характеристики:

- професійні навички, що відображають технології та інструменти, якими володіє розробник;
- спеціалізація, яка визначає напрям професійної діяльності;
- роль у проєкті, що характеризує функції та відповідальність розробника;
- предметна область, яка відображає досвід роботи у певній сфері;
- рівень освіти;
- попередній професійний досвід.

У системі передбачається можливість збереження кількох значень для окремих характеристик, зокрема навичок, ролей, спеціалізацій та предметних областей, що дозволяє більш точно описати професійний профіль розробника.

Модель розробника подається у вигляді сукупності взаємопов'язаних сутностей, де центральним елементом є розробник, а інші сутності описують його характеристики. Для наочного представлення структури моделі розробника використовується схема взаємозв'язків між елементами моделі (рис. 4.1)

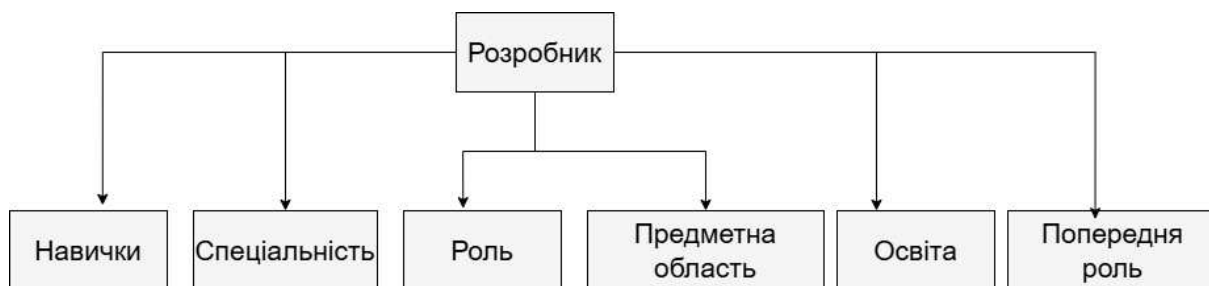


Рисунок 4.1 – Структура моделі розробника

У системі також визначається модель програмної задачі, яка використовується для формування вимог до кандидата.

Модель задачі описує характеристики, необхідні для визначення вимог до виконавця, та включає такі елементи:

- назва задачі;
- опис задачі;
- тип задачі;
- пріоритет;
- тривалість виконання;
- необхідний рівень досвіду;
- необхідна спеціалізація;
- необхідні ролі;
- необхідні професійні навички;
- рівень володіння навичками;
- вагові коефіцієнти параметрів (weight), що визначають важливість кожного критерію;
- необхідний рівень освіти;
- предметна область.

Усі зазначені характеристики використовуються для формування формалізованих вимог до кандидата.

Особливістю моделі задачі є використання вагових коефіцієнтів для кожного параметра, що дозволяє враховувати різну важливість критеріїв під час підбору кандидатів.

Модель задачі представлена у вигляді набору взаємопов'язаних сутностей, що дозволяє зберігати структуру вимог до задачі та використовувати її під час виконання алгоритму підбору. Для наочного представлення структури моделі задачі використовується схема взаємозв'язків між елементами моделі (рис. 4.2).

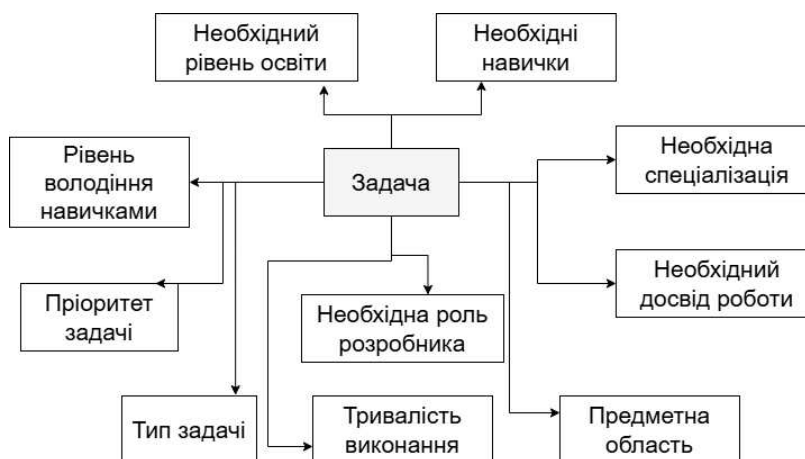


Рисунок 4.2 – Структура моделі задачі програмного проєкту

Модель розробника та модель задачі є взаємопов'язаними та будуть використовуватися спільно під час реалізації процесу підбору кандидатів.

На основі характеристик задачі формується набір вимог до кандидата, після чого здійснюватиметься зіставлення цих вимог із характеристиками розробників.

Використання формалізованих моделей дозволяє:

- представити інформацію про розробників у структурованому вигляді;
- забезпечити об'єктивне порівняння кандидатів;
- застосовувати кількісні методи оцінювання відповідності;
- реалізувати автоматизований підбір і ранжування кандидатів.

Таким чином, побудова моделей розробника та задачі є основою для подальшої реалізації алгоритмів підбору кандидатів у вебзастосунку.

4.2 Архітектура вебзастосунку

Розроблюваний вебзастосунок призначений для зберігання інформації про розробників, опису програмних задач та автоматизованого підбору кандидатів для їх виконання. Для забезпечення зручності розробки, підтримки та розширення системи використовується багаторівнева архітектура застосунку.

Архітектура системи включає кілька основних рівнів, кожен із яких виконує певні функції та відповідає за обробку різних типів даних.

Першим рівнем архітектури вебзастосунку є користувацький інтерфейс. Він формується із використанням технології Razor Pages платформи .NET [8], а також стандартних вебтехнологій: HTML, CSS, Bootstrap і JavaScript.

Технологія Razor Pages забезпечує зручну організацію вебзастосунку за рахунок поєднання розмітки та серверної логіки. Кожна сторінка складатиметься з файлу розмітки (.cshtml), який містить HTML-код інтерфейсу, та файлу моделі сторінки (.cshtml.cs), у якому здійснюватиметься обробка запитів, робота з даними та взаємодія з бізнес-логікою застосунку.

HTML використовується для формування структури вебсторінок, CSS і Bootstrap для стилізації інтерфейсу та забезпечення адаптивного відображення. Це дозволяє створювати зручні та зрозумілі форми для введення даних користувачем.

JavaScript використовується для реалізації динамічної поведінки інтерфейсу та підвищення зручності роботи із системою. Зокрема, передбачається можливість динамічного додавання полів введення без повного перезавантаження сторінки (наприклад, під час додавання нових навичок розробника або вимог до задачі), а також обробка дій користувача та взаємодія з елементами сторінки.

Таким чином, користувацький інтерфейс забезпечує зручну взаємодію користувача із системою, введення даних та перегляд результатів підбору кандидатів.

Другим рівнем є рівень прикладної логіки. На цьому рівні визначатиметься основні алгоритми роботи системи, зокрема алгоритм аналізу відповідності розробників вимогам задач та алгоритм підбору кандидатів. Основна логіка підбору

передбачається у вигляді сервісу `MatchingService`, який здійснює аналіз характеристик розробників, обчислення рейтингу кандидатів і формування списку найбільш відповідних фахівців.

Наступним рівнем є рівень доступу до даних. Він відповідає за взаємодію застосунку з базою даних. Для роботи з даними використовується технологія `Entity Framework Core` [9], яка дозволяє виконувати операції створення, читання, оновлення та видалення даних у базі.

Останнім рівнем архітектури є база даних. У базі даних передбачається зберігання інформації про розробників, їхні професійні навички, ролі, спеціалізації, предметні області, а також інформації про програмні задачі та вимоги до кандидатів.

Взаємодія між рівнями архітектури здійснюватиметься за допомогою HTTP-запитів, які надходять від користувача через вебінтерфейс. Після отримання запиту серверна частина обробляє його, звертається до рівня прикладної логіки, який, у разі необхідності, взаємодіє з базою даних через `Entity Framework Core`. Результати обробки будуть передаватися назад у користувацького інтерфейсу та відображатимуться у вигляді вебсторінок.

Вибір технології `Razor Pages` зумовлений її тісною інтеграцією з платформою `.NET`, що дозволяє спростити розробку серверної логіки та забезпечити зручну організацію коду. Використання `Entity Framework Core` дає можливість ефективно працювати з базою даних, зменшити обсяг ручного SQL-коду та підвищити продуктивність розробки. Обрана архітектура забезпечує баланс між простотою реалізації та можливістю подальшого розширення системи.

Використання багаторівневої архітектури дозволяє розділити відповідальність між різними частинами системи, спростити розробку та підвищити зручність супроводження застосунку. Такий підхід робить систему більш гнучкою та дозволяє за необхідності розширювати її функціональність.

Для наочного представлення структури вебзастосунку використовується архітектурна схема (рис. 4.3), яка відображає взаємодію основних рівнів системи.

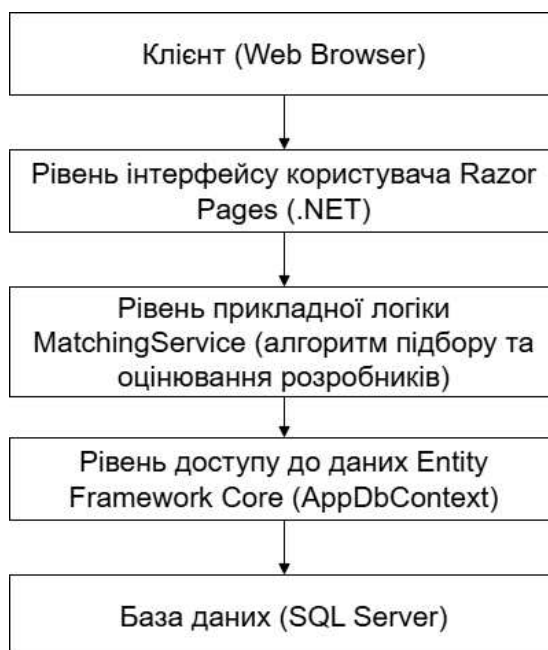


Рисунок 4.3 – Архітектура вебзастосунку

4.3 Проєктування структури бази даних

У межах проєктування вебзастосунку підбору розробників виконується розробка структури бази даних, призначеної для зберігання інформації про розробників, їхні професійні характеристики, а також програмні задачі та вимоги до кандидатів. Для реалізації зберігання даних передбачається використання реляційної бази даних.

У розроблюваному вебзастосунку передбачається використання системи керування базами даних SQLite [11]. Для взаємодії застосунку з базою даних використовується технологія Entity Framework Core, яка забезпечує об'єктно-реляційне відображення даних і дозволяє виконувати операції створення, читання, оновлення та видалення записів.

Структура бази даних передбачає використання кількох основних таблиць, які відображають характеристики розробників і вимоги програмних задач.

Центральною таблицею бази даних є таблиця Developers, яка міститиме основну інформацію про розробників. У цій таблиці передбачається зберігання таких даних, як ім'я розробника, рівень освіти, кількість років професійного досвіду та поточна роль у проєкті.

Для зберігання інформації про професійні навички передбачено використання таблиць Skills і DeveloperSkills. Таблиця Skills міститиме перелік технологій та інструментів розробки, а таблиця DeveloperSkills забезпечує зв'язок між розробниками та їхніми навичками і дозволяє зберігати рівень володіння кожною технологією.

Інформація про спеціалізації розробників зберігається в таблицях Specialities і DeveloperSpecialities, які дозволяють визначити професійний напрям діяльності розробника.

Для зберігання інформації про ролі розробників використовується таблиця Roles, а таблиця DeveloperRoles забезпечує зберігання даних про ролі, які розробник виконував раніше у проєктах.

Предметні області розробки будуть представлені в таблиці Domains, а зв'язок між розробниками та предметними областями будуть представлені за допомогою таблиці DeveloperDomains.

Для зберігання інформації про програмні задачі передбачається використання таблиці Tasks, яка міститиме основні параметри задачі та вимоги до кандидатів. Вимоги до навичок, ролей і спеціалізацій розробників реалізовуватимуться за допомогою таблиць TaskRequirements, TaskRoles і TaskSpecialities, які забезпечують зв'язок задач із відповідними характеристиками розробників.

Між таблицями бази даних передбачено встановлення зв'язків різних типів. Зокрема, зв'язки типу один-до-багатьох (1:N) будуть використовуватися між основними сутностями, наприклад між розробниками та їхніми характеристиками. Для реалізації зв'язків багато-до-багатьох (N:M) будуть застосовуватись допоміжні таблиці, такі як DeveloperSkills, DeveloperRoles, DeveloperSpecialities та TaskRequirements, що дозволяє гнучко описувати взаємозв'язки між сутностями.

Використання зовнішніх ключів забезпечує цілісність даних та узгодженість зв'язків між таблицями, що є важливим для коректної роботи алгоритму підбору кандидатів.

Таким чином, спроектована структура бази даних дозволяє зберігати детальну інформацію про розробників і вимоги програмних задач, а також

забезпечує можливість ефективного аналізу відповідності кандидатів вимогам проекту.

Для наочного представлення структури бази даних використовується ER-діаграма, яка відображає сутності системи та зв'язки між ними (рис. 4.3).

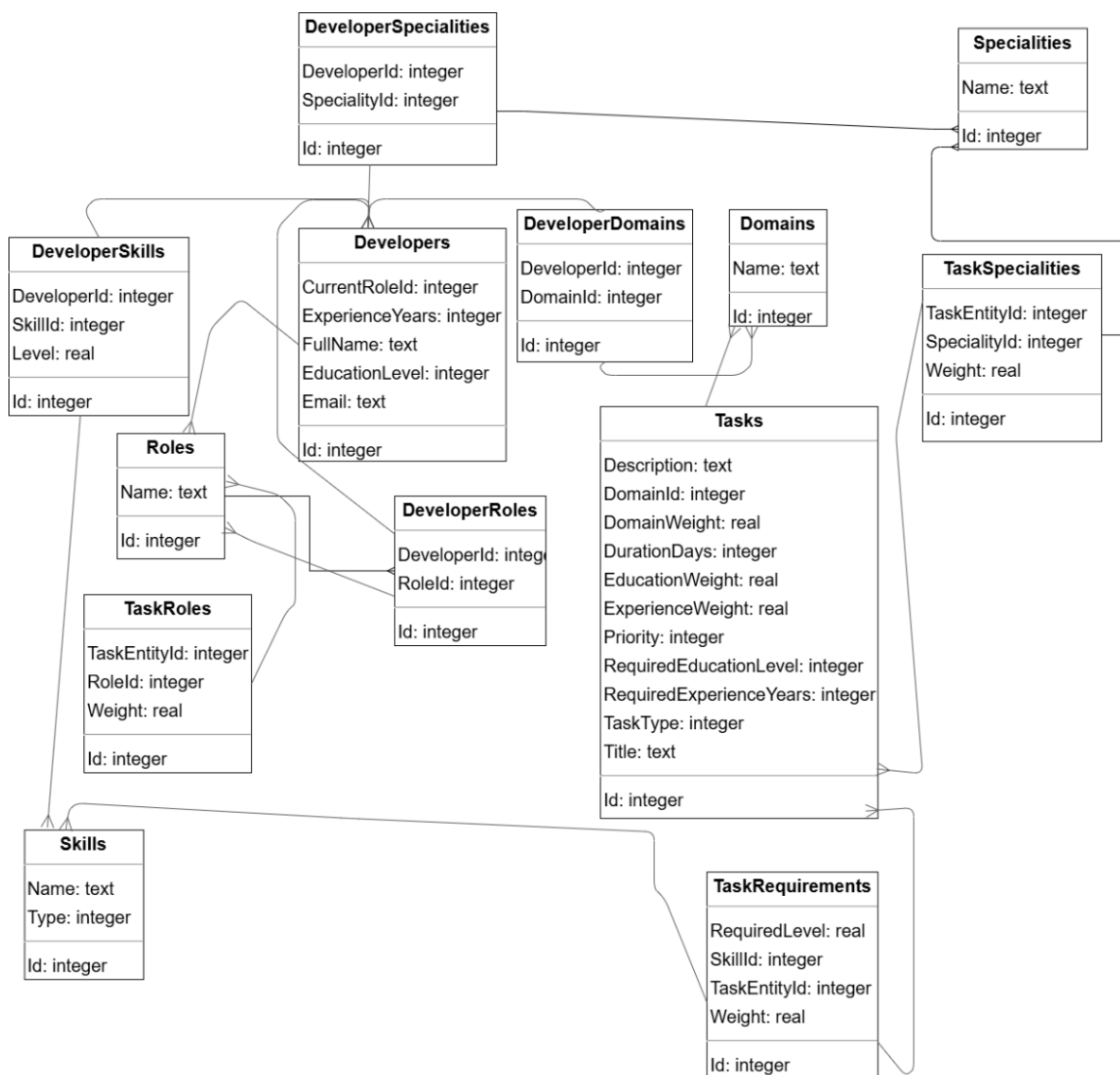


Рисунок 4.3 – ER-діаграма бази даних вебзастосунку

Висновки до розділу 4

У четвертому розділі роботи було виконано проектування вебзастосунку для підбору розробників до виконання програмних задач.

Було розроблено архітектуру вебзастосунку на основі багаторівневого

підходу, який передбачає розділення системи на рівень інтерфейсу користувача, рівень прикладної логіки та рівень доступу до даних. Такий підхід забезпечує гнучкість, масштабованість та зручність подальшого супроводження і розширення системи.

Окрему увагу приділено проектуванню структури реляційної бази даних, яка забезпечує зберігання інформації про розробників, їхні професійні характеристики, а також параметри програмних задач і вимоги до кандидатів. Спроектована структура бази даних дозволяє ефективно організувати взаємозв'язки між сутностями та забезпечує цілісність даних.

Таким чином, у результаті виконаного проектування було сформовано цілісну модель вебзастосунку, яка створює основу для подальшої реалізації системи та забезпечує можливість ефективного підбору розробників відповідно до заданих вимог.

5 ПРОГРАМНА РЕАЛІЗАЦІЯ ВЕБЗАСТОСУНКУ

5.1 Вибір інструментів розробки

Розробка вебзастосунку для підбору розробників здійснюється з використанням сучасного технологічного стеку, що забезпечує надійність, масштабованість і зручність супроводу системи.

Як основну платформу для реалізації серверної частини застосунку обрано технологію .NET, мова програмування C# з використанням Razor Pages. Такий підхід дозволяє об'єднати розмітку та серверну логіку в межах однієї сторінки, що спрощує структуру проєкту та підвищує зручність розробки. Razor Pages забезпечує обробку HTTP-запитів, маршрутизацію, роботу з формами та взаємодію з бізнес-логікою застосунку.

Для реалізації взаємодії з базою даних використовується технологія Entity Framework Core, яка виконує функції об'єктно-реляційного відображення (ORM). Це дозволяє працювати з даними на рівні об'єктів, використовуючи класи та LINQ-запити, що значно спрощує виконання операцій створення, читання, оновлення та видалення даних. Як система управління базами даних обрано SQLite легковагову реляційну СУБД, оптимально придатну для локального розгортання та тестування застосунку.

Клієнтська частина вебзастосунку реалізована з використанням стандартних вебтехнологій: HTML, CSS і JavaScript. Для створення адаптивного користувацького інтерфейсу використовується бібліотека Bootstrap, що дозволяє забезпечити коректне відображення застосунку на різних пристроях. JavaScript застосовується для реалізації динамічної поведінки інтерфейсу, включаючи обробку дій користувача та зміну вмісту сторінок без їх повного перезавантаження.

Розробка застосунку виконується в середовищі JetBrains Rider, яке надає зручні інструменти для роботи з проєктами на платформі .NET, включаючи інтелектуальне автодоповнення коду, навігацію по проєкту, інтеграцію із системою контролю версій і засоби аналізу якості коду.

Таким чином, обраний набір технологій дозволить реалізувати функціональний, гнучкий і зручний у використанні вебзастосунок для підбору розробників.

5.2 Реалізація сервісу підбору кандидатів

Сервіс підбору кандидатів є ключовим компонентом розробленого вебзастосунку, оскільки саме він реалізує основну функціональність системи автоматизований вибір розробників, які найбільше відповідають вимогам програмної задачі. Реалізація алгоритму підбору кандидатів безпосередньо базується на математичній моделі, представлений у розділі 3.

Сервіс реалізовано у вигляді окремого класу `MatchingService` (Додаток Г), який взаємодіє з базою даних через контекст `AppDbContext` (Додаток В) із використанням технології `Entity Framework Core`. Це дозволяє ефективно працювати з взаємопов'язаними сутностями та виконувати необхідні операції вибірки даних.

Приклади програмної реалізації ключових моделей системи наведені у додатках:

- у Додатку А Програмна реалізація моделі `Developer`;
- у Додатку Б Програмна реалізація моделі `TaskEntity`.

Основним методом сервісу є асинхронний метод `FindBestCandidatesAsync`, який реалізує повний цикл підбору кандидатів.

На початковому етапі виконується завантаження задачі разом з усіма пов'язаними даними, а також формується список розробників. Для підвищення ефективності алгоритму реалізовано попередню фільтрацію: до подальшого аналізу допускаються лише ті кандидати, які мають хоча б одну з необхідних навичок задачі.

Лістинг 5.1 – Завантаження даних і попередня фільтрація

```
var task = await _context.Tasks
    .Include(t => t.TaskSkills).ThenInclude(ts => ts.Skill)
```

```

.Include(t => t.TaskRoles).ThenInclude(tr => tr.Role)
.Include(t => t.TaskSpecialities).ThenInclude(ts => ts.Speciality)
.Include(t => t.Domain)
.FirstOrDefaultAsync(t => t.Id == taskId);

var requiredSkillIds = task.TaskSkills
    .Where(ts => ClampWeight(ts.Weight) > 0)
    .Select(s => s.SkillId)
    .Distinct()
    .ToHashSet();

developersQuery = developersQuery
    .Where(d => d.DeveloperSkills.Any(ds =>
        requiredSkillIds.Contains(ds.SkillId)));

```

Даний фрагмент демонструє початковий етап роботи сервісу підбору кандидатів. На цьому етапі виконується завантаження задачі разом із її вимогами, а також здійснюється попередня фільтрація розробників. До вибірки включаються лише ті кандидати, які мають хоча б одну з необхідних навичок, що дозволяє значно скоротити обсяг обчислень і підвищити ефективність алгоритму.

Лістинг 5.2 – Оцінка відповідності навичок

```

f = devSkill.Level / req.RequiredLevel.Value;
f = Math.Min(f, 1.0);
f = Clamp01(f);

```

Даний фрагмент демонструє реалізацію оцінювання відповідності навичок розробника вимогам задачі. Розрахунок виконується відповідно до формули (3.1), де значення визначається як відношення фактичного рівня володіння навичкою до необхідного рівня. Отримане значення нормалізується в діапазоні від 0 до 1, що забезпечує коректне використання цього показника в підсумковій моделі оцінювання.

Лістинг 5.3 – Оцінка відповідності досвіду

```

var f = (double)developer.ExperienceYears / task.RequiredExperienceYears.Value;
f = Math.Min(f, 1.0);
f = Clamp01(f);

```

Аналогічний підхід застосовується для оцінювання відповідності досвіду роботи розробника вимогам задачі. Розрахунок виконується відповідно до формули

(3.2), де враховується співвідношення фактичного досвіду роботи розробника та необхідного досвіду, заданого в параметрах задачі.

Після формування списку кандидатів для кожного розробника виконується обчислення ступеня відповідності з використанням методу CalculateScore.

Лістинг 5.4 – Фрагмент методу CalculateScore

```
var totalWeight = factors.Sum(f => f.Weight);

if (totalWeight <= 0)
    return 0;

var score = factors.Sum(f => f.Contribution) / totalWeight;

return Clamp01(score);
```

Даний фрагмент демонструє завершальний етап обчислення ступеня відповідності кандидата. У методі CalculateScore для кожного критерію формуються окремі фактори оцінювання, які надалі агрегуються у підсумковий результат. Розрахунок виконується як середньозважене значення факторів відповідності, що відповідає формулі (3.3), поданий у попередньому розділі.

Лістинг 5.5 – Розрахунок показника Skill Surplus

```
var diff = devSkill.Level - req.RequiredLevel.Value;
if (diff > 0)
    surplus += diff;
```

Даний фрагмент реалізує обчислення показника перевищення рівня навичок розробника (Skill Surplus). Розрахунок відповідає формулі (3.4) та відображає сумарне перевищення фактичного рівня навичок над необхідними значеннями. Цей показник використовується як додатковий критерій під час ранжування кандидатів.

Після обчислення всіх показників формується підсумковий список результатів, який містить інформацію про розробників, їх рейтинг, фактори оцінювання та додаткові показники.

Сортування кандидатів виконується за такими критеріями:

- за спаданням підсумкового рейтингу;

- за сумарним внеском факторів;
- за значенням показника Skill Surplus.

Такий підхід дозволяє забезпечити більш точне ранжування кандидатів, особливо у випадках, коли значення рейтингу є близькими.

Повна програмна реалізація сервісу наведена у Додатку Г.

Висновки до розділу 5

У п'ятому розділі було розглянуто програмну реалізацію вебзастосунку підбору кандидатів. Зокрема, обґрунтовано вибір технологій розробки, серед яких платформа .NET, технологія Razor Pages, Entity Framework Core та СКБД SQLite. Обраний стек технологій забезпечує зручність розробки, ефективну взаємодію з базою даних та можливість подальшого розширення системи.

Основну увагу приділено реалізації сервісу підбору кандидатів, який є ключовим компонентом системи. Було реалізовано алгоритм оцінювання відповідності розробників вимогам задачі на основі математичної моделі, представленої у попередньому розділі.

Реалізація алгоритму включає попередню фільтрацію кандидатів, обчислення факторів відповідності за окремими критеріями та визначення підсумкового рейтингу як середньозваженого значення. Додатково враховується показник перевищення рівня навичок, що дозволяє більш точно ранжувати кандидатів.

Таким чином, у результаті виконаної програмної реалізації було створено функціональний механізм підбору кандидатів, який забезпечує автоматизоване оцінювання та впорядкування розробників відповідно до заданих вимог.

6 ТЕСТУВАННЯ ВЕБЗАСТОСУНКУ

6.1 Тестування функціональних вимог

Метою тестування є перевірка коректності реалізації основних функціональних вимог вебзастосунок підбору розробників.

Тестування проводилося на основі сценаріїв використання системи, сформульованих на етапі формування вимог. Основна увага приділялася перевірці ключових функцій застосунку, пов'язаних із додаванням розробників, створенням задач і виконанням підбору кандидатів.

Перевірка виконувалася вручну шляхом взаємодії з користувацьким інтерфейсом застосунку. У процесі тестування послідовно виконувалися основні дії користувача та аналізувалися отримані результати.

Результати тестування наведені в таблиці 6.1.

Таблиця 6.1 – Результати тестування функціональних вимог

ID кейсу	Назва функціоналу	Вхідні дані	Кроки виконання	Очікуваний результат	Фактичний результат	Статус
1	2	3	4	5	6	7
TC01	Додавання розробника	Дані розробника (ім'я, досвід, навички)	Заповнити форму та зберегти	Розробник доданий у систему	Відповідає очікуваному	Пройдено
TC02	Додавання навичок	Навички та рівень володіння	Додати навички розробнику	Навички зберігаються коректно	Відповідає очікуваному	Пройдено
TC03	Створення задачі	Назва, опис задачі	Заповнити форму задачі	Задача створюється та зберігається	Відповідає очікуваному	Пройдено
TC04	Задання вимог	Навички, рівень, вага	Додати вимоги до задачі	Вимоги зберігаються	Відповідає очікуваному	Пройдено

Кінець таблиці 6.1

1	2	3	4	5	6	7
TC05	Запуск підбору	Обрана задача	Запустити і підбір кандидатів	Виконується підбір	Відповідає очікуваному	Пройдено
TC06	Попередня фільтрація	Розробники без необхідних навичок	Запустити і підбір	Відбираються релевантні кандидати	Відповідає очікуваному	Пройдено
TC07	Розрахунок рейтингу	Дані розробників і задачі	Виконати підбір	Розраховується рейтинг	Відповідає очікуваному	Пройдено
TC08	Сортування кандидатів	Список кандидатів	Виконати підбір	Кандидати відсортовані за рейтингом	Відповідає очікуваному	Пройдено
TC09	Відображення результатів	Результати підбору	Відкрити сторінку результатів	Відображається список кандидатів	Відповідає очікуваному	Пройдено
TC10	Відсутність кандидатів	Несумісні вимоги	Запустити і підбір	Список кандидатів порожній	Відповідає очікуваному	Пройдено

У межах тестування було перевірено коректність роботи основних функцій вебзастосунку. Тестування охоплює ключові сценарії взаємодії користувача із системою, включаючи додавання розробників, формування задач і виконання підбору кандидатів. Усі перевірені функції успішно пройшли тестування, що підтверджує коректність реалізації бізнес-логіки системи та відповідність розробленого застосунку поставленим функціональним вимогам.

На рисунку 6.1 наведено візуалізацію покриття функціональних вимог

тестами.



Рисунок 6.1 – Графік покриття функціональних вимог тестами

6.2 Тестування нефункціональних вимог

Нефункціональні вимоги визначають не лише набір функцій, що виконує система, але й характеристики якості її роботи, такі як продуктивність, зручність використання, стійкість і коректність обробки даних.

Тестування нефункціональних характеристик вебзастосунку підбору розробників проводилося з метою оцінювання якості роботи системи в умовах, наближених до реальної експлуатації. Основна увага приділялася таким аспектам: продуктивність, зручність користувацького інтерфейсу, стійкість роботи та коректність обробки помилок.

Продуктивність системи оцінювалася шляхом виконання типових операцій, таких як створення задач, додавання розробників і запуск підбору кандидатів. У ході перевірки встановлено, що система обробляє запити без помітних затримок, а час відгуку залишається на прийнятному рівні за умов звичайного навантаження.

Зручність користувацького інтерфейсу перевірялася в процесі взаємодії із системою. Інтерфейс застосунку є інтуїтивно зрозумілим, забезпечує просту навігацію та дозволяє користувачу без ускладнень виконувати основні дії.

Стійкість системи оцінювалася під час виконання послідовних операцій і

повторних запитів. У ході тестування не було виявлено збоїв або некоректної поведінки застосунку.

Коректність обробки помилок перевірялася шляхом введення некоректних або неповних даних. У таких випадках система коректно обробляє помилки та запобігає виконанню некоректних операцій.

Результати тестування наведені в таблиці 6.2.

Таблиця 6.2 – Результати тестування нефункціональних вимог

№	Вимога	Метод перевірки	Результат перевірки	Статус
1	Продуктивність	Виконання операцій створення та підбору	Час відгуку без помітних затримок	Успішно
2	Зручність інтерфейсу (UX)	Ручна взаємодія із системою	Інтерфейс зрозумілий, навігація зручна	Успішно
3	Стійкість роботи	Повторне виконання операцій	Система працює стабільно	Успішно
4	Обробка помилок	Введення некоректних даних	Помилки коректно обробляються	Успішно
5	Коректність даних	Перевірка збереження та відображення	Дані зберігаються та відображаються коректно	Успішно
6	Робота алгоритму	Багаторазовий запуск підбору	Результати стабільні та відтворювані	Успішно

Результати тестування показують, що розроблений вебзастосунок відповідає основним нефункціональним вимогам. Система демонструє стабільну роботу, коректно обробляє дії користувача та забезпечує прийнятну продуктивність під час виконання основних операцій.

Інтерфейс застосунку є зручним у використанні, а реалізований алгоритм

підбору кандидатів працює стабільно та видає відтворювані результати. Це підтверджує якість реалізації системи та її готовність до практичного застосування.

Висновки до розділу 6

У шостому розділі було проведено тестування розробленого вебзастосунку підбору кандидатів з метою перевірки коректності його роботи та відповідності поставленим вимогам.

У процесі тестування функціональних вимог перевірено основні сценарії взаємодії користувача із системою, зокрема додавання розробників, створення задач, налаштування вимог та виконання підбору кандидатів. Отримані результати підтвердили коректність реалізації функціоналу та правильність роботи алгоритму підбору.

Тестування нефункціональних характеристик показало, що система забезпечує стабільну роботу, прийнятну швидкодію та зручність використання інтерфейсу. Обробка помилок виконується коректно, що підвищує надійність роботи застосунку.

Таким чином, результати тестування підтверджують, що розроблений вебзастосунок відповідає основним функціональним і нефункціональним вимогам та є придатним для практичного використання.

7 ЕКСПЛУАТАЦІЯ ЗАСТОСУНКУ

7.1 Робота користувача з системою

Після відкриття застосунку користувач потрапляє на головну сторінку системи, де доступні основні розділи для роботи з даними (рис. 7.1).

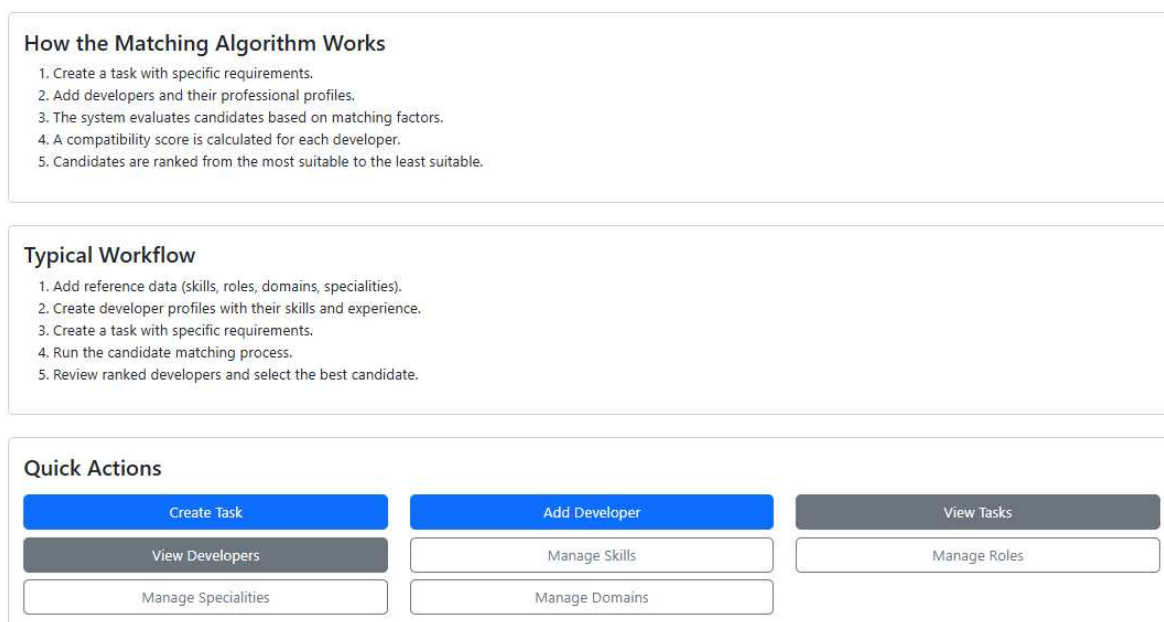


Рисунок 7.1 – Головна сторінка вебзастосунку

Користувач може перейти до розділу управління розробниками, де представлено список усіх доданих фахівців (рис. 7.2).

У даному розділі користувач має можливість:

- додавати нового розробника;
- редагувати інформацію про розробника;
- видаляти розробника із системи.

Це дозволяє ефективно керувати інформацією про фахівців та підтримувати актуальність даних у системі.

Developers

Create Developer

Name	Email	Current Role	Experience	Education	Previous Roles	Specialities	Domains	Skills	Actions
Alex Johnson	alexjohnson@gmail.com	Senior Backend Developer	5 years	Master	Junior Backend Developer Middle Backend Developer	Web development	FinTech	C# (Level: 9) ASP NET (Level: 9) SQL (Level: 9)	<a>Edit <a>Delete
Mark Lee	marklee@gmail.com	Middle Backend Developer	3 years	Bachelor	Junior Backend Developer	Software Engineering	E-Commerce	C# (Level: 7) ASP NET (Level: 6) SQL (Level: 1)	<a>Edit <a>Delete
John Doe	johndoe@gmail.com	Junior Fullstack Developer	1 years	Bachelor	Junior Backend Developer	Information Systems	Education	python (Level: 5)	<a>Edit <a>Delete
Enid Sinclair	enidsinclair@gmail.com	Junior Backend Developer	1 years	None	Junior Backend Developer	Computer Science	Gaming	C# (Level: 6)	<a>Edit <a>Delete
Max Brown	maxbrown@gmail.com	Backend developer	7 years	Master	Middle Backend Developer Senior Backend Developer Backend developer	Software Engineering Web development	FinTech	C# (Level: 9) ASP NET (Level: 9) SQL (Level: 8) docker (Level: 5)	<a>Edit <a>Delete
Mary Sunderland	marys@gmail.com	Middle Fullstack Developer	4 years	Bachelor	Junior Backend Developer Middle Backend Developer	Web development	FinTech	C# (Level: 7) ASP NET (Level: 7) SQL (Level: 6) javascript (Level: 5)	<a>Edit <a>Delete

Рисунок 7.2 – Список розробників у системі

Під час створення розробника користувач заповнює форму, у якій зазначає основні характеристики, такі як ім'я, досвід роботи, рівень освіти, а також додає навички, ролі, спеціалізації та предметні області (рис. 7.3).

Інтерфейс форми створення розробника забезпечує зручне та послідовне введення даних, дозволяючи користувачу заповнювати як основну інформацію, так і додаткові характеристики фахівця.

Завдяки використанню динамічних елементів інтерфейсу користувач може додавати необхідну кількість навичок без перезавантаження сторінки, що підвищує зручність роботи із системою.

Усі введені дані зберігаються у системі та використовуються для подальшого аналізу відповідності розробників вимогам задач.

Add Developer

Basic Information

FullName

Current Role

ExperienceYears

EducationLevel

Specialities

Computer Science
Software Engineering
Computer Engineering
Information Systems

Domains

FinTech
E-Commerce
Education
Gaming

Previous Roles

Junior Backend Developer
Middle Backend Developer
Senior Backend Developer
Junior Fullstack Developer

Hard Skills

Add at least one skill so the system can understand developer's profile.

Add Hard Skill

Рисунок 7.3 – Форма додавання розробника

Аналогічно реалізовано розділ управління задачами, у якому користувач може переглядати список задач, створювати нові задачі, редагувати та видаляти існуючі (рис. 7.4).

Інтерфейс розділу забезпечує наочне представлення задач та дозволяє користувачу оперативно виконувати основні дії з ними.

Кожна задача відображається у вигляді структурованого запису з основними параметрами, що дозволяє швидко оцінити її характеристики. Це спрощує навігацію по списку задач та підвищує зручність їх подальшого опрацювання.

Tasks

[+ Create Task](#)

Note:

Required Level (1–10) – minimum skill proficiency expected from a candidate.

Weight (0–10) – defines how important this requirement is when calculating the best match.

Higher weight values have a greater impact on the final candidate score.

Basic Info	Roles	Specialities	Skill Requirements	Actions
Backend API for Payment System Type: Backend Priority: High Duration: 30 days Domain: FinTech Domain weight: 5 Required experience: 3 years Experience weight: 6 Required education: Bachelor Education weight: 2	Middle Backend Developer – Weight: 5 Senior Backend Developer – Weight: 3	Software Engineering – Weight: 2 Web development – Weight: 4	C# – Level: 8 – Weight: 8 ASP NET – Level: 7 – Weight: 7 SQL – Level: 6 – Weight: 5	Edit Delete Find candidates
Build scalable REST API for fintech platform Type: Backend Priority: Medium Duration: 15 days Domain: FinTech Domain weight: 7 Required experience: 5 years Experience weight: 6 Required education: Bachelor Education weight: 2	Backend developer – Weight: 7 Fullstack developer – Weight: 5	Web development – Weight: 8 Software Engineering – Weight: 6	C# – Level: 8 – Weight: 10 ASP NET – Level: 8 – Weight: 10 SQL – Level: 6 – Weight: 7 docker – Level: 3 – Weight: 4	Edit Delete Find candidates

Рисунок 7.4 – Список задач

Інтерфейс створення задачі забезпечує зручне введення всіх необхідних параметрів та дозволяє користувачу гнучко налаштовувати вимоги до кандидатів. Завдяки цьому формується структурований опис задачі, який використовується в подальшому для автоматизованого підбору розробників.

Застосування вагових коефіцієнтів дає можливість визначати пріоритетність окремих критеріїв під час оцінювання кандидатів. Це дозволяє системі більш точно враховувати вимоги задачі та формувати обґрунтований рейтинг розробників.

Введені параметри задачі зберігаються у системі та надалі використовуються як основа для аналізу відповідності розробників заданим вимогам.

Під час створення задачі користувач задає параметри, включаючи опис задачі, необхідні навички, ролі, спеціалізації, рівень досвіду та освіти, а також зазначає ваги критеріїв (рис. 7.5).

Create Task

Basic Info

Title

Description

Task Type

Priority

DurationDays

Required Experience (years)

ExperienceWeight ⓘ

Domain

DomainWeight ⓘ

Required Education

RequiredEducationLevel

EducationWeight ⓘ

Roles

Add roles and set how important each is for this task (0–10).

+ Add Role

Specialities

Add specialities and set how important each is for this task (0–10).

+ Add Speciality

Skill Requirements

Add at least one skill so the system can find the best matching specialist. Skills with **weight = 0** are informational only and do not affect matching or candidate filtering.

Note:

Candidates without any matching required skill will be excluded from the selection process.

+ Add Skill

Create Task

Рисунок 7.5 – Форма додавання задачі

Однією з ключових функцій системи є автоматизований підбір кандидатів для виконання задачі. Для цього користувач у списку задач обирає необхідну задачу та натискає кнопку «Find Candidates». Після цього система виконує аналіз відповідності розробників вимогам задачі та формує список найбільш відповідних

кандидатів (рис. 7.6).

Best Candidates

Develop AI-based recommendation service

Develop a backend service that generates personalized recommendations using machine learning models. The system must process large datasets, build prediction models, and expose the results through a REST API. The developer will work with Python, machine learning libraries, and database technologies to build a scalable recommendation system.

Requirements

Level describes the minimum proficiency expected for a skill (1–10).

Weight (0–10) shows how important this requirement is for the final matching score: 0 means it does not affect the score, 10 means it has a very strong impact.

- **Type:** Backend, **Priority:** Medium, **Duration:** 20 days
- **Experience:** at least 4 years (weight: 5)
- **Domain:** AI (weight: 6)
- **Required education:** Master (weight: 3)

Roles (number in brackets is weight): Backend developer (5) Junior Backend Developer (5) Middle Backend Developer (5) Senior Backend Developer (5) Data Scientist (8)

Specialities (number in brackets is weight): Machine Learning (10) Data Engineering (5)

Required skills:

- python - required level: 7, weight: 10
- Machine learning - required level: 6, weight: 9
- SQL - required level: 5, weight: 0
- docker - required level: 4, weight: 3

Matching rule:

Candidates without any matching required skill will be excluded from the selection process.

Rank	Developer	Score (0..1)	Experience	Education	Current Role	Skill surplus ^①	
1	Daniel Kim	0.747	6	PhD	Data Scientist	7	Details
2	Robert Jackson	0.487	7	Bachelor	Backend developer	2	Details
3	Max Brown	0.329	7	Master	Backend developer	1	Details
4	John Doe	0.170	1	Bachelor	Junior Fullstack Developer	0	Details

[Back to Tasks](#)

Рисунок 7.6 – Запуск підбору кандидатів

Додатково в системі реалізована можливість управління довідковими даними, які використовуються під час формування характеристик розробників і вимог задач. До таких даних належать ролі, професійні навички, спеціалізації та предметні області.

Управління ролями (Roles). Користувач може перейти до розділу управління ролями, де представлено список усіх доступних ролей розробників.

У даному розділі доступні такі операції:

- додавання нової ролі;
- редагування існуючої ролі;
- видалення ролі із системи.

Під час додавання нової ролі система виконує перевірку на унікальність. У разі, якщо роль із таким самим найменуванням уже існує, додавання дублікату не допускається, що забезпечує цілісність даних (рис. 7.8).

Roles	
Create New Role	
Name	Actions
Junior Backend Developer	Edit Delete
Middle Backend Developer	Edit Delete
Senior Backend Developer	Edit Delete

Рисунок 7.8 – Розділ управління ролями

Управління навичками (Skills). У системі передбачено окремий розділ для управління професійними навичками розробників.

Користувач може:

- додавати нові навички;
- редагувати існуючі;
- видаляти навички.

Під час додавання нового навика система перевіряє, чи існує вже навик із таким самим найменуванням. Це дозволяє уникнути дублювання та забезпечує коректність подальшого аналізу відповідності розробників вимогам задач (рис. 7.9).

Skills		
Add Skill		
Name	Type	Actions
C#	HardSkill	Edit Delete
javascript	HardSkill	Edit Delete
go lang	HardSkill	Edit Delete

Рисунок 7.9 – Розділ управління навичками

Управління спеціалізаціями (Specialities). Розділ спеціалізацій призначений для опису професійних напрямів діяльності розробників.

У даному розділі користувач може:

- створювати нові спеціалізації;
- редагувати їх;
- видаляти за потреби.

Як і в інших довідниках, під час додавання виконується перевірка на унікальність значень, що виключає появу однакових записів у системі (рис. 7.10).

Specialities

Create New Speciality

Name	Actions
Computer Science	Edit Delete
Software Engineering	Edit Delete
Computer Engineering	Edit Delete

Рисунок 7.10 – Розділ управління спеціалізаціями

Управління предметними областями (Domain). Предметні області використовуються для визначення сфери застосування проєкту та досвіду розробника.

Користувач може:

- додавати нові предметні області;
- редагувати існуючі;
- видаляти їх із системи.

Інтерфейс управління предметними областями забезпечує наочне відображення наявних записів та зручний доступ до основних операцій. Користувач має можливість швидко переглядати список предметних областей, а також виконувати дії редагування або видалення безпосередньо зі списку.

Система також контролює унікальність значень, запобігаючи додаванню повторюваних записів (рис. 7.11).

Domains	
Create New Domain	
Name	Actions
FinTech	Edit Delete
E-Commerce	Edit Delete
Education	Edit Delete

Рисунок 7.11 – Розділ управління предметними областями

Окрім зазначеного, у системі реалізовано механізм забезпечення цілісності даних під час видалення довідкових сутностей.

У разі, якщо користувач намагається видалити роль, спеціалізацію, навичку або предметну область, яка вже пов’язана з певною задачею або розробником, виконання цієї операції блокується. У такому випадку на сторінці відображається відповідне повідомлення, у якому зазначається, що обрана сутність використовується у системі.

Повідомлення також містить інформацію про пов’язані об’єкти, зокрема:

- перелік задач, у яких використовується дана сутність;
- перелік розробників, до яких вона прив’язана.

Для виконання операції видалення користувачу необхідно попередньо усунути всі наявні зв’язки, тобто видалити відповідні прив’язки до задач або розробників. Лише після цього система дозволяє виконати видалення сутності.

Такий підхід забезпечує узгодженість даних та запобігає виникненню помилок, пов’язаних із порушенням зв’язків між об’єктами системи.

Користувач отримує детальну інформацію про наявні зв’язки, що перешкоджають видаленню, що дозволяє прийняти обґрунтоване рішення щодо подальших дій. Це сприяє запобіганню втраті даних та забезпечує коректність функціонування системи.

Приклади відображення відповідних повідомлень наведено на рисунках 7.12–7.15.

Delete Role

Deletion is only possible after you detach this role from all related developers and tasks listed below.

This role cannot be deleted.

It is still used by the following developers and/or tasks:

Developers:

- [Mark Lee](#)
- [John Doe](#)
- [Mary Sunderland](#)
- [Alex Johnson](#)

Tasks:

- [Develop AI-based recommendation service](#)

Please remove these links first and then try deleting the role again.

Junior Backend Developer

Delete

Cancel

Рисунок 7.12 – Повідомлення про неможливість видалення ролі

Delete Skill

Warning!

Deletion is only possible after you detach this skill from all related developers and tasks listed below.

This skill cannot be deleted.

It is still used by the following developers and/or tasks:

Developers:

- [Mark Lee](#)
- [Enid Sinclair](#)
- [Max Brown](#)
- [Mary Sunderland](#)
- [James Sunderland](#)
- [Alex Johnson](#)

Tasks:

- [Backend API for Payment System](#)
- [Build scalable REST API for fintech platform](#)

Please remove these links first and then try deleting the skill again.

Are you sure you want to delete:

C#

Delete

Cancel

Рисунок 7.13 – Повідомлення про неможливість видалення навички

Delete Speciality

Deletion is only possible after you detach this speciality from all related developers and tasks listed below.

This speciality cannot be deleted.

It is still used by the following developers and/or tasks:

Developers:

- [Mark Lee](#)
- [Max Brown](#)

Tasks:

- [Backend API for Payment System](#)
- [Build scalable REST API for fintech platform](#)

Please remove these links first and then try deleting the speciality again.

Software Engineering

Delete

Cancel

Рисунок 7.14 – Повідомлення про неможливість видалення спеціалізації

Delete Domain

Deletion is only possible after you detach this domain from all related developers and tasks listed below.

This domain cannot be deleted.

It is still used by the following developers and/or tasks:

Developers:

- [Max Brown](#)
- [Mary Sunderland](#)
- [James Sunderland](#)
- [Sophie Martin](#)
- [Alex Johnson](#)

Tasks:

- [Backend API for Payment System](#)
- [Build scalable REST API for fintech platform](#)

Please remove these links first and then try deleting the domain again.

FinTech

Delete

Cancel

Рисунок 7.15 – Повідомлення про неможливість видалення предметної області

7.2 Приклад підбору кандидатів для програмної задачі

У даному підрозділі розглядається приклад роботи системи підбору кандидатів для конкретної програмної задачі з використанням розробленого вебзастосунку.

На сторінці підбору кандидатів додатково представлено пояснювальну інформаційну панель, яка допомагає користувачу коректно інтерпретувати параметри задачі.

Зокрема, в інтерфейсі відображаються визначення ключових характеристик:

- Level – мінімальний рівень володіння навичкою, необхідний для виконання задачі (у діапазоні від 1 до 10);
- Weight – показник значущості вимоги під час розрахунку підсумкового рейтингу кандидата (у діапазоні від 0 до 10), де 0 означає відсутність впливу на результат, а 10 – максимальний вплив.

Наявність цих пояснень дозволяє користувачу краще розуміти, яким чином формуються вимоги до кандидатів і як окремі параметри впливають на підсумкову оцінку відповідності.

Крім того, на сторінці у зручній структурованій формі представлено всю інформацію про задачу, включаючи її опис, вимоги, ролі, спеціалізації та навички, що забезпечує наочність і спрощує аналіз умов підбору кандидатів.

Як приклад обрано задачу «Build React analytics dashboard», для якої необхідно розробити інтерактивну аналітичну панель із використанням технології React та сучасних інструментів фронтенд-розробки.

На сторінці підбору кандидатів (рис. 7.16) відображається повна інформація про задачу, включаючи вимоги до кандидатів:

- необхідні навички із зазначенням необхідного рівня та ваги;
- вимоги до досвіду роботи;
- предметна область;
- рівень освіти;
- ролі та спеціалізації.

Також на сторінці зазначено правило підбору (Matching rule): кандидати, які не володіють жодною з необхідних навичок, автоматично виключаються з процесу відбору.

Best Candidates

Build React analytics dashboard

Create an interactive analytics dashboard using React and modern frontend technologies. The dashboard should display real-time metrics and integrate with backend APIs.

Requirements

Level describes the minimum proficiency expected for a skill (1–10).

Weight (0–10) shows how important this requirement is for the final matching score: 0 means it does not affect the score, 10 means it has a very strong impact.

- **Type:** Frontend, **Priority:** Low, **Duration:** 20 days
- **Experience:** at least 3 years (weight: 3)
- **Domain:** E-Commerce (weight: 4)
- **Required education:** Bachelor (weight: 1)

Roles (number in brackets is weight): Frontend developer (9) Fullstack developer (6)

Specialities (number in brackets is weight): Web development (8)

Required skills:

- javascript - required level: 7, weight: 9
- react - required level: 7, weight: 10
- html - required level: 6, weight: 7
- css - required level: 6, weight: 7

Matching rule:

Candidates without any matching required skill will be excluded from the selection process.

Rank	Developer	Score (0..1)	Experience	Education	Current Role	Skill surplus ^①	
1	Sophie Martin	0.906	5	Bachelor	Frontend developer	6	Details
2	James Sunderland	0.641	6	Bachelor	Fullstack developer	0	Details
3	Mary Sunderland	0.288	4	Bachelor	Middle Fullstack Developer	0	Details

[Back to Tasks](#)

Рисунок 7.16 – Сторінка підбору кандидатів для програмної задачі

Після виконання алгоритму система формує список кандидатів, відсортованих за спаданням підсумкового рейтингу.

У розглянутому прикладі системою було знайдено трьох кандидатів:

- Sophie Martin – рейтинг 0.906;
- James Sunderland – рейтинг 0.641;
- Mary Sunderland – рейтинг 0.288.

Рейтинг кандидата набуває значення в діапазоні від 0 до 1 і розраховується на основі зваженої оцінки відповідності вимогам задачі.

На першому місці знаходиться кандидат Sophie Martin із рейтингом 0.906 (рис. 7.17).

Rank	Developer	Score (0..1)	Experience	Education	Current Role	Skill surplus ①	
1	Sophie Martin	0.906	5	Bachelor	Frontend developer	6	Details
Score breakdown							
Factor	Weight	Match (0..1)	Contribution	Details			
Skill: react	10	1	10	Dev level: 8, required: 7			
Skill: javascript	9	1	9	Dev level: 8, required: 7			
Role: Frontend developer	9	1	9	Role matches (current or previous).			
Speciality: Web development	8	1	8	Speciality matches.			
Skill: html	7	1	7	Dev level: 8, required: 6			
Skill: css	7	1	7	Dev level: 8, required: 6			
Role: Fullstack developer	6	0	0	Role does not match.			
Domain: E-Commerce	4	1	4	Domain matches.			
Experience (years)	3	1	3	Dev: 5, required: 3			
Education ≥ Bachelor	1	1	1	Dev: Bachelor			
Weighted sum: 58, Total weight: 64, Skill surplus: 6 (total levels above required across all required skills)							

Рисунок 7.17 – Деталізація оцінювання кандидата Sophie Martin

Високий рейтинг даного кандидата пояснюється такими факторами:

- кандидат повністю відповідає вимогам за ключовими навичками (React, JavaScript, HTML, CSS), що відповідає значенню 1 за формулою (3.1);
- рівень навичок кандидата перевищує необхідний, що відображається у показнику Skill Surplus = 6 (формула (3.4));
- кандидат відповідає необхідній ролі (Frontend developer) та спеціалізації;
- повністю відповідає вимогам щодо досвіду роботи та рівня освіти;
- співпадає предметна область задачі.

Таким чином, більшість факторів мають максимальні значення, що

призводить до високого підсумкового рейтингу, обчисленого за формулою (3.3).

Кандидат James Sunderland займає друге місце з рейтингом 0.641 (рисуюнок 7.18).

Score breakdown

Factor	Weight	Match (0..1)	Contribution	Details
Skill: react	10	1	10	Dev level: 7, required: 7
Skill: javascript	9	1	9	Dev level: 7, required: 7
Role: Frontend developer	9	0	0	Role does not match.
Speciality: Web development	8	1	8	Speciality matches.
Skill: html	7	0	0	Candidate does not have this skill.
Skill: css	7	0	0	Candidate does not have this skill.
Role: Fullstack developer	6	1	6	Role matches (current or previous).
Domain: E-Commerce	4	1	4	Domain matches.
Experience (years)	3	1	3	Dev: 6, required: 3
Education ≥ Bachelor	1	1	1	Dev: Bachelor

Weighted sum: 41, Total weight: 64, Skill surplus: 0 (total levels above required across all required skills)

Рисуюнок 7.18 – Деталізація оцінювання кандидата James Sunderland

Незважаючи на відповідність ряду вимог, його рейтинг є нижчим з таких причин:

- відсутня відповідність ролі Frontend developer (значення 0);
- відсутні деякі навички (HTML, CSS), що знижує внесок у підсумковий рейтинг;
- показник Skill Surplus дорівнює 0, оскільки рівень навичок не перевищує необхідний.

Таким чином, часткова невідповідність вимогам призводить до зниження підсумкової оцінки.

Кандидат Mary Sunderland має найнижчий рейтинг 0.288 (рисуюнок 7.19).

Отримане значення рейтингу свідчить про недостатню відповідність кандидата заданим вимогам, що відображається у структурі підсумкової оцінки.

3

Mary Sunderland

0.288

4

Bachelor

Middle Fullstack Developer

0

Score breakdown

Factor	Weight	Match (0..1)	Contribution	Details
Skill: react	10	0	0	Candidate does not have this skill.
Skill: javascript	9	0.714	6.429	Dev level: 5, required: 7
Role: Frontend developer	9	0	0	Role does not match.
Speciality: Web development	8	1	8	Speciality matches.
Skill: html	7	0	0	Candidate does not have this skill.
Skill: css	7	0	0	Candidate does not have this skill.
Role: Fullstack developer	6	0	0	Role does not match.
Domain: E-Commerce	4	0	0	Domain does not match.
Experience (years)	3	1	3	Dev: 4, required: 3
Education ≥ Bachelor	1	1	1	Dev: Bachelor

Weighted sum: 18.429, Total weight: 64, Skill surplus: 0 (total levels above required across all required skills)

Back to Tasks

Рисунок 7.19 – Деталізація оцінювання кандидата Mary Sunderland

Низький рейтинг пояснюється такими факторами:

- відсутній ключовий навик React (значення 0);
- рівень володіння JavaScript нижчий за необхідний (формула (3.1));
- відсутні навички HTML та CSS;
- не відповідає необхідна роль і предметна область;
- відсутнє перевищення рівня навичок (Skill Surplus = 0).

У результаті більшість факторів мають низькі значення, що призводить до мінімального підсумкового рейтингу.

Розглянутий приклад показує, що розроблений алгоритм підбору кандидатів коректно оцінює ступінь відповідності розробників вимогам задачі.

Система дозволяє:

- автоматично ранжувати кандидатів;
- враховувати значущість кожного критерію за рахунок використання ваг;
- отримувати детальне пояснення результатів підбору;
- виявляти сильні та слабкі сторони кандидатів.

Таким чином, реалізований алгоритм забезпечує обґрунтований і прозорий вибір найбільш відповідних розробників для виконання програмних задач.

Висновки до розділу 7

У цьому розділі було розглянуто роботу розробленого вебзастосунку для підбору кандидатів до виконання програмних задач.

Було описано процес взаємодії користувача із системою, включаючи роботу з основними сутностями, такими як розробники, задачі, навички, ролі, спеціалізації та предметні області. Показано, що система забезпечує зручний користувацький інтерфейс і підтримує виконання основних операцій з управління даними.

Розглянуто приклад підбору кандидатів для конкретної програмної задачі з використанням реалізованого алгоритму. Продемонстровано, що система коректно виконує фільтрацію кандидатів, враховує задані вимоги та формує відсортований список розробників відповідно до ступеня їх відповідності.

Показано, що використання деталізованої структури оцінювання дозволяє аналізувати внесок кожного критерію в підсумковий рейтинг кандидата. Це забезпечує прозорість роботи алгоритму та можливість інтерпретації отриманих результатів.

Таким чином, розроблений вебзастосунок забезпечує наочну демонстрацію роботи алгоритму підбору кандидатів і підтверджує його ефективність при вирішенні задачі вибору розробників для програмних проєктів.

ЗАГАЛЬНІ ВИСНОВКИ

У даній кваліфікаційній роботі вирішено задачу розробки вебзастосунку, призначеного для побудови моделі розробника програмного забезпечення та автоматизованого підбору кандидатів для виконання програмних задач.

На початковому етапі роботи було проведено аналіз вимог підбору розробників, що дозволило визначити основні проблеми та складність даного процесу. Виконано аналіз існуючих аналогічних рішень, у результаті якого виявлено їхні переваги та недоліки, а також обґрунтовано необхідність створення власної системи. У межах роботи було сформовано функціональні та нефункціональні вимоги до вебзастосунку, визначено основні варіанти використання системи та сценарії взаємодії користувача з нею, проведено оцінювання ризиків і визначено ключові фактори, що впливають на ефективність підбору кандидатів.

Було виконано формалізацію предметної області, у межах якої визначено основні характеристики розробника, включаючи професійні навички, спеціалізацію, ролі в проєкті, предметну область, досвід роботи та рівень освіти. На цій основі розроблено структуровану модель розробника, що дозволяє представити професійний профіль фахівця у формалізованому вигляді.

Було розроблено модель програмної задачі, яка включає опис вимог до кандидата з урахуванням різних параметрів і їх значущості. Використання вагових коефіцієнтів дозволяє враховувати різну важливість критеріїв під час підбору розробників.

Ключовим результатом роботи є розробка алгоритму підбору кандидатів, який включає етапи формування вимог, попередньої фільтрації, оцінювання відповідності та ранжування розробників. Для оцінювання відповідності використано математичну модель, що базується на нормалізації параметрів і обчисленні підсумкового рейтингу кандидата у вигляді зваженого значення. Додатково введено показник перевищення рівня навичок, що дозволяє більш точно розрізняти кандидатів із близькими значеннями рейтингу.

На основі розроблених моделей і алгоритмів реалізовано вебзастосунок із використанням сучасних технологій, включаючи платформу .NET, мову програмування C#, Razor Pages, Entity Framework Core та систему управління базами даних SQLite. Реалізований застосунок забезпечує можливість управління даними про розробників і задачі, а також автоматизований підбір кандидатів з подальшим відображенням результатів.

У процесі тестування було підтверджено, що система коректно реалізує задані функціональні та нефункціональні вимоги. Результати тестування показали стабільну роботу застосунку, коректну обробку користувацьких даних і адекватне функціонування алгоритму підбору.

Практична цінність розробленої системи полягає в автоматизації процесу підбору розробників, зниженні трудомісткості аналізу кандидатів і підвищенні об'єктивності прийняття рішень. Запропонований підхід дозволяє враховувати велику кількість факторів одночасно та формувати обґрунтований рейтинг кандидатів.

Таким чином, поставлені в роботі задачі було повністю виконано, а розроблений вебзастосунок може бути використаний як інструмент підтримки прийняття рішень при підборі розробників у межах програмних проєктів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Nana Assyne, Hadi Ghanbari, Mirja Pulkkinen. The essential competencies of software professionals: A unified competence framework. *Information and Software Technology*, Volume 151, 2022, 107020. <https://doi.org/10.1016/j.infsof.2022.107020> (дата звернення: 03.02.2026).
2. Кунгурцев, О., & Чорба, Р. (2025). Метод автоматизованого вибору виконавця на основі моделі завдання з розробки програмного забезпечення. *Automation of Technological and Business Processes*, 17(3), 115-124. <https://doi.org/10.15673/atbp.v17i3.3262> (дата звернення: 03.02.2026).
3. Oleksii Kungurtsev, Radim Chorba. A term-dictionary-based technology for selecting task executors in software development project. *Romanian Journal of Information Technology and Automatic Control*. ISSN 1220-1758, vol. 35(4), pp. 21-33, 2025. <https://doi.org/10.33436/v35i4y202502> (дата звернення: 03.02.2026).
4. Чорба Р.В. (2025) Підвищення ефективності на основі використання моделі спеціаліста. Наук.кер Кунгурцев О.Б. XV міжнародна наукова конференція молодих вчених. Сучасні Інформаційні Технології. 15-16 травня 2025 <https://doi.org/10.5281/zenodo.15521809> (дата звернення: 03.02.2026).
5. Upwork. URL: <https://www.upwork.com> (дата звернення: 05.02.2026).
6. LinkedIn. URL: <https://www.linkedin.com> (дата звернення: 05.02.2026).
7. GitHub. URL: <https://github.com> (дата звернення: 05.02.2026).
8. Microsoft. .NET documentation. URL: <https://learn.microsoft.com/dotnet> (дата звернення: 05.03.2026).
9. Microsoft Entity Framework Core documentation. URL: <https://learn.microsoft.com/ef/core> (дата звернення: 11.03.2026).
10. Microsoft ASP.NET Core documentation. URL: <https://learn.microsoft.com/aspnet/core> (дата звернення: 10.03.2026).
11. SQLite Documentation. URL: <https://www.sqlite.org/docs.html> (дата звернення: 11.03.2026).
12. Кунгурцев О.Б., Скрипнікова А., Чорба Р.В. (2026). Реалізація моделі

розробника програмного забезпечення. І Міжнародна науково-практична конференція «Інформаційні та операційні технології в судоплавстві. (прийнято до публікації)

ДОДАТОК А ПРОГРАМНА РЕАЛІЗАЦІЯ МОДЕЛІ DEVELOPER

```

using System.ComponentModel.DataAnnotations;
using DeveloperSelectionSystem.Data.Enums;

namespace DeveloperSelectionSystem.Data.Entities;

public class Developer
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Full name is required")]
    [StringLength(100, ErrorMessage = "Full name cannot exceed 100 characters")]
    public string FullName { get; set; } = string.Empty;

    [Required(ErrorMessage = "Experience is required")]
    [Range(0, 50, ErrorMessage = "Experience must be between 0 and 50 years")]
    public int ExperienceYears { get; set; }

    public int? CurrentRoleId { get; set; }
    public Role? CurrentRole { get; set; }

    public EducationLevel EducationLevel { get; set; }

    public ICollection<DeveloperSkill> DeveloperSkills { get; set; } = new
    List<DeveloperSkill>();

    public ICollection<DeveloperDomain> DeveloperDomains { get; set; } = new
    List<DeveloperDomain>();

    public ICollection<DeveloperRole> PreviousRoles { get; set; } = new
    List<DeveloperRole>();

    public ICollection<DeveloperSpeciality> DeveloperSpecialities { get; set; } =
    new List<DeveloperSpeciality>();
}

```

ДОДАТОК Б ПРОГРАМНА РЕАЛІЗАЦІЯ МОДЕЛІ TASKENTITY

```

using System.ComponentModel.DataAnnotations;
using DeveloperSelectionSystem.Data.Enums;

namespace DeveloperSelectionSystem.Data.Entities;

public class TaskEntity
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Please enter task title")]
    [StringLength(200, ErrorMessage = "Title cannot exceed 200 characters")]
    public string Title { get; set; } = string.Empty;

    [Required(ErrorMessage = "Please enter task description")]
    [StringLength(1000, ErrorMessage = "Description cannot exceed 1000
characters")]
    public string Description { get; set; } = string.Empty;

    [Required(ErrorMessage = "Please select task type")]
    public TaskType TaskType { get; set; }

    [Required(ErrorMessage = "Please select task priority")]
    public TaskPriority Priority { get; set; }

    [Range(1, 3650, ErrorMessage = "Please enter task duration in days (at least
1).")]
    public int DurationDays { get; set; }

    [Range(0, 60, ErrorMessage = "Please enter non-negative required experience in
years.")]
    public int? RequiredExperienceYears { get; set; }

    [Range(0,10)]
    public double? ExperienceWeight { get; set; }

    public int? DomainId { get; set; }
    public Domain? Domain { get; set; }

    [Range(0,10)]
    public double? DomainWeight { get; set; }

    public EducationLevel? RequiredEducationLevel { get; set; }

    [Range(0,10)]
    public double? EducationWeight { get; set; }
    public ICollection<TaskSkill> TaskSkills { get; set; } = new
List<TaskSkill>();

    public ICollection<TaskSpeciality> TaskSpecialities { get; set; } = new
List<TaskSpeciality>();

    public ICollection<TaskRole> TaskRoles { get; set; } = new List<TaskRole>();
}

```

ДОДАТОК В КОНФІГУРАЦІЯ КОНТЕКСТУ БАЗИ ДАНИХ

APPDBCONTEXT

```

using Microsoft.EntityFrameworkCore;
using DeveloperSelectionSystem.Data.Entities;

namespace DeveloperSelectionSystem.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options)
        {
            public DbSet<Developer> Developers { get; set; }
            public DbSet<Skill> Skills { get; set; }
            public DbSet<DeveloperSkill> DeveloperSkills { get; set; }
            public DbSet<TaskEntity> Tasks { get; set; }
            public DbSet<TaskSkill> TaskRequirements { get; set; }

            public DbSet<Domain> Domains { get; set; }

            public DbSet<DeveloperDomain> DeveloperDomains { get; set; }

            public DbSet<Speciality> Specialities { get; set; }
            public DbSet<DeveloperSpeciality> DeveloperSpecialities { get; set; }
            public DbSet<TaskSpeciality> TaskSpecialities { get; set; }

            public DbSet<Role> Roles { get; set; }
            public DbSet<DeveloperRole> DeveloperRoles { get; set; }

            public DbSet<TaskRole> TaskRoles { get; set; }
        }
    }
}

```

ДОДАТОК Г РЕАЛІЗАЦІЯ СЕРВІСУ ПІДБОРУ КАНДИДАТІВ

```

using DeveloperSelectionSystem.Data;
using DeveloperSelectionSystem.Data.Entities;
using DeveloperSelectionSystem.Data.Enums;
using Microsoft.EntityFrameworkCore;

namespace DeveloperSelectionSystem.Services.Matching;

public class MatchingService : IMatchingService
{
    private readonly AppDbContext _context;

    public MatchingService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<MatchResult>> FindBestCandidatesAsync(int taskId, int?
top = null)
    {
        var task = await _context.Tasks
            .Include(t => t.TaskSkills).ThenInclude(ts => ts.Skill)
            .Include(t => t.TaskRoles).ThenInclude(tr => tr.Role)
            .Include(t => t.TaskSpecialities).ThenInclude(ts => ts.Speciality)
            .Include(t => t.Domain)
            .FirstOrDefaultAsync(t => t.Id == taskId);

        if (task == null)
            return new List<MatchResult>();

        // --- Попередня фільтрація за навичками: кандидат повинен мати хоча б
одну ОБОВ'ЯЗКОВУ навичку.
        // "Обов'язкова" у цьому випадку означає, що її вага > 0 (навички з вагою
0 вважаються лише інформаційними).
        var requiredSkillIds = task.TaskSkills
            .Where(ts => ClampWeight(ts.Weight) > 0)
            .Select(s => s.SkillId)
            .Distinct()
            .ToHashSet();

        // Завантаження розробників разом з усіма пов'язаними даними, необхідними
для підбору.
        // Якщо існують вимоги до навичок з ненульовою вагою, виконується
фільтрація:
        // розробник повинен мати хоча б ОДНУ з цих навичок (за SkillId).
        // Рівень навички на цьому етапі НЕ перевіряється – кандидати з нижчим
рівнем
        // залишаються у вибірці, але отримають нижчу оцінку.
        var developersQuery = _context.Developers
            .AsNoTracking()
            .Include(d => d.DeveloperSkills).ThenInclude(ds => ds.Skill)
            .Include(d => d.DeveloperDomains).ThenInclude(dd => dd.Domain)
            .Include(d => d.PreviousRoles).ThenInclude(pr => pr.Role)
            .Include(d => d.DeveloperSpecialities).ThenInclude(sp =>
sp.Speciality)
            .Include(d => d.CurrentRole)
            .AsQueryable();

        if (requiredSkillIds.Any())

```

```

    {
        developersQuery = developersQuery
            .Where(d => d.DeveloperSkills.Any(ds =>
requiredSkillIds.Contains(ds.SkillId)));
    }

    var developers = await developersQuery.ToListAsync();

    var results = new List<MatchResult>();

    foreach (var dev in developers)
    {
        var score = CalculateScore(task, dev, out var factors);

        var surplus = CalculateSkillSurplus(task, dev);

        // Якщо totalWeight == 0, то підсумковий рейтинг дорівнює 0
        results.Add(new MatchResult
        {
            Developer = dev,
            Score = score,
            Factors = factors,
            SkillSurplus = surplus
        });
    }

    var ordered = results
        .OrderByDescending(r => r.Score)
        .ThenByDescending(r => r.WeightedSum)
        .ThenByDescending(r => r.SkillSurplus)
        .ToList();

    if (top.HasValue && top.Value > 0)
        ordered = ordered.Take(top.Value).ToList();

    return ordered;
}

public double CalculateScore(TaskEntity task, Developer developer, out
List<MatchFactorResult> factors)
{
    factors = new List<MatchFactorResult>();

    // Навички (кожна навичка задачі робить внесок у підсумкову оцінку)
    foreach (var req in task.TaskSkills)
    {
        var weight = ClampWeight(req.Weight);

        // Якщо користувач задає вагу 0, це не впливає на підсумковий рейтинг,
тому така вимога пропускається
        // if (weight <= 0) continue;
        var devSkill = developer.DeveloperSkills.FirstOrDefault(s => s.SkillId
== req.SkillId);

        // Якщо кандидат не має цієї навички, значення дорівнює 0 (але
кандидат не виключається, оскільки відбір вже виконано за умовою наявності хоча б
однієї спільної навички)
        double f = 0;
        if (devSkill != null && req.RequiredLevel.HasValue &&
req.RequiredLevel.Value > 0)
        {

```

```

        f = devSkill.Level / req.RequiredLevel.Value;
        f = Math.Min(f, 1.0); // cap at 1
        f = Clamp01(f);
    }

    factors.Add(new MatchFactorResult
    {
        Name = $"Skill: {req.Skill?.Name ?? req.SkillId.ToString()}",
        Weight = weight,
        Value = f,
        Details = devSkill == null
            ? "Candidate does not have this skill."
            : $"Dev level: {devSkill.Level:0.##}, required:
{req.RequiredLevel:0.##}"
    });
}

// Досвід
// Передбачається, що TaskEntity містить поля: RequiredExperienceYears та
ExperienceWeight
    if (task.RequiredExperienceYears.HasValue &&
task.RequiredExperienceYears.Value > 0)
    {
        var weight = ClampWeight(task.ExperienceWeight ?? 0);
        if (weight > 0)
        {
            var f = (double)developer.ExperienceYears /
task.RequiredExperienceYears.Value;
            f = Math.Min(f, 1.0);
            f = Clamp01(f);

            factors.Add(new MatchFactorResult
            {
                Name = "Experience (years)",
                Weight = weight,
                Value = f,
                Details = $"Dev: {developer.ExperienceYears}, required:
{task.RequiredExperienceYears}"
            });
        }
    }

// Предметна область
// Передбачається, що TaskEntity містить поля DomainId та DomainWeight,
// а розробник має колекцію DeveloperDomains (зв'язок багато-до-багатьох)
if (task.DomainId.HasValue && task.DomainId.Value > 0)
{
    var weight = ClampWeight(task.DomainWeight ?? 0);
    if (weight > 0)
    {
        var hasDomain = developer.DeveloperDomains.Any(d => d.DomainId ==
task.DomainId.Value);
        factors.Add(new MatchFactorResult
        {
            Name = $"Domain: {task.Domain?.Name ??
task.DomainId.Value.ToString()}",
            Weight = weight,
            Value = hasDomain ? 1.0 : 0.0,
            Details = hasDomain ? "Domain matches." : "Domain does not
match."
        });
    }
}

```

```

    }
}

// Освіта
// Передбачається, що TaskEntity містить поля RequiredEducationLevel
(nullable) та EducationWeight
var requiredEdu = task.RequiredEducationLevel;
if (requiredEdu.HasValue)
{
    var weight = ClampWeight(GetDoubleProperty(task, "EducationWeight"));
    if (weight > 0)
    {
        var ok = developer.EducationLevel >= requiredEdu.Value;
        factors.Add(new MatchFactorResult
        {
            Name = $"Education ≥ {requiredEdu.Value}",
            Weight = weight,
            Value = ok ? 1.0 : 0.0,
            Details = $"Dev: {developer.EducationLevel}"
        });
    }
}

// Ролі
// Кожна вимога до ролі враховується окремо з власною вагою
(TaskRole.Weight)
foreach (var reqRole in task.TaskRoles)
{
    var weight = ClampWeight(reqRole.Weight);
    if (weight <= 0) continue;

    var hasRole =
        (developer.CurrentRoleId.HasValue && developer.CurrentRoleId.Value
== reqRole.RoleId) ||
        developer.PreviousRoles.Any(r => r.RoleId == reqRole.RoleId);

    factors.Add(new MatchFactorResult
    {
        Name = $"Role: {reqRole.Role?.Name ?? reqRole.RoleId.ToString()}",
        Weight = weight,
        Value = hasRole ? 1.0 : 0.0,
        Details = hasRole ? "Role matches (current or previous)." : "Role
does not match."
    });
}

// Спеціалізації
foreach (var reqSpec in task.TaskSpecialities)
{
    var weight = ClampWeight(reqSpec.Weight);
    if (weight <= 0) continue;

    var hasSpec = developer.DeveloperSpecialities.Any(s => s.SpecialityId
== reqSpec.SpecialityId);

    factors.Add(new MatchFactorResult
    {
        Name = $"Speciality: {reqSpec.Speciality?.Name ??
reqSpec.SpecialityId.ToString()}",
        Weight = weight,
        Value = hasSpec ? 1.0 : 0.0,

```



```

        Details = hasSpec ? "Speciality matches." : "Speciality does not
match."
    });
}

var totalWeight = factors.Sum(f => f.Weight);
if (totalWeight <= 0)
    return 0;

var score = factors.Sum(f => f.Contribution) / totalWeight;
return Clamp01(score);
}

// ----- допоміжні методи -----

// Сума перевищення рівня навичок розробника відносно необхідного рівня
// для всіх вимог до навичок (враховуються лише навички з вагою > 0)
private static double CalculateSkillSurplus(TaskEntity task, Developer
developer)
{
    double surplus = 0;

    foreach (var req in task.TaskSkills)
    {
        var weight = ClampWeight(req.Weight);
        if (weight <= 0) continue; // ігнорувати інформаційні вимоги

        if (!req.RequiredLevel.HasValue || req.RequiredLevel.Value <= 0)
            continue;

        var devSkill = developer.DeveloperSkills.FirstOrDefault(s => s.SkillId
== req.SkillId);
        if (devSkill == null) continue;

        var diff = devSkill.Level - req.RequiredLevel.Value;
        if (diff > 0)
            surplus += diff;
    }

    return surplus;
}

private static double Clamp01(double x) => x < 0 ? 0 : (x > 1 ? 1 : x);

// вага повинна бути в діапазоні 0..10 (межі можна змінювати)
private static double ClampWeight(double x)
{
    if (double.IsNaN(x)) return 0;
    if (x < 0) return 0;
    if (x > 10) return 10;
    return x;
}

// Безпечне зчитування необов'язкових числових властивостей з TaskEntity
// (щоб сервіс не падав, якщо якась властивість була перейменована)
private static double GetDoubleProperty(TaskEntity task, string propertyName)
{
    var prop = task.GetType().GetProperty(propertyName);
    if (prop == null) return 0;

    var val = prop.GetValue(task);

```

```

        if (val == null) return 0;

        return val switch
        {
            double d => d,
            float f => f,
            int i => i,
            decimal m => (double)m,
            _ => 0
        };
    }

    private static EducationLevel? GetNullableEducation(TaskEntity task, string
propertyName)
    {
        var prop = task.GetType().GetProperty(propertyName);
        if (prop == null)
            return null;

        var val = prop.GetValue(task);
        if (val == null)
            return null;

        if (val is EducationLevel edu)
            return edu;

        return null;
    }
}

```